

Lecture: 15

Termination Detection



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed the basic concepts of '**Distributed Minimum Spanning Tree**' and also discuss the algorithms *i.e.* GHS algorithm.

Content of this Lecture:

- In this lecture, we will discuss about '**Termination Detection**' and set of representative termination detection algorithms based on the concepts of **snapshot collection, weight throwing and spanning-tree etc.**

Introduction

- The **termination detection problem** was brought to prominence in **1980** by **‘Francez and by Dijkstra and Scholten’**.
- A fundamental problem:** To determine if a distributed computation has terminated.
- A non-trivial task** since no process has complete knowledge of the global state, and global time does not exist.
- A distributed computation is globally terminated if every process is locally terminated and there is no message in transit between any processes.
- “Locally terminated”** state is a state in which a process has finished its computation and will not restart any action unless it receives a message.
- In the termination detection problem, a particular process (or all of the processes) must infer when the underlying computation has terminated.

Introduction

- A termination detection algorithm is used for this purpose.
- Messages used in the underlying computation are called '**basic messages**', and messages used for the purpose of termination detection are called '**control messages**'.
- A **termination detection (TD) algorithm** must ensure the following:
 1. Execution of a TD algorithm cannot indefinitely delay the underlying computation.
 2. The termination detection algorithm must not require addition of new communication channels between processes.

System Model

- At any given time, a process can be in only one of the two states: **active**, where it is doing local computation and **idle**, where the process has (temporarily) finished the execution of its local computation and will be reactivated only on the receipt of a message from another process.
- An active process can become idle at any time.
- An idle process can become active only on the receipt of a message from another process.
- Only active processes can send messages.
- A message can be received by a process when the **process is in either of the two states, i.e., active or idle**. On the receipt of a message, an idle process becomes active.
- The sending of a message and the receipt of a message occur as atomic actions.

Definition of Termination Detection

- Let $p_i(t)$ denote the **state (active or idle)** of **process p_i** at **instant t** .
- Let $c_{i,j}(t)$ denote the **number of messages in transit in the channel** at **instant t** from **process p_i** to **process p_j** .
- A distributed computation is said to be terminated at time instant t_0 **iff: $(\forall i :: p_i(t_0) = \text{idle}) \wedge (\forall i, j :: c_{i,j}(t_0) = 0)$** .
- Thus, a distributed computation has terminated iff all processes have become idle and there is no message in transit in any channel.

Termination Detection Algorithms

Termination Detection Algorithms

Authors	Year	Algorithm
Dijkstra et al.	1983	Ring-based termination detection algorithm.
R. W. Topor	1984	Termination detection for distributed computations
F. Mattern	1987	Developed several algorithms for termination detection for the atomic model of computation.
Shing-Tsaan Huang	1989	Termination detection algorithm that uses distributed snapshot
Shing-Tsaan Huang	1989	Termination detection algorithm based on weight throwing
F. Mattern	1989	Termination detection algorithm based on weight throwing
Chandrasekaran and Venkatesan	1990	Message optimal termination detection algorithm
Brzezinski <i>et al.</i>	1993	Develop algorithms to detect static and dynamic terminations.
Yu-Chee Tseng	1995	Termination detection under faulty processes
J. Mayo and P. Kearns	1994, 1995	Efficient termination detection based on roughly synchronized clocks

1. Termination detection Using Distributed Snapshots By “ST Huang” (1989)

- The algorithm assumes that there is a **logical bidirectional communication channel** between every pair of processes.
- Communication channels are **reliable but non-FIFO**. Message delay is arbitrary but finite.

Main idea:

- When a process goes from active to idle, it issues a request to all other processes to take a local snapshot, and also requests itself to take a local snapshot.
- When a process receives the request, if it agrees that the requester became idle before itself, it grants the request by taking a local snapshot for the request.
- A request is **successful** if all processes have taken a local snapshot for it.
- The requester or any external agent may collect all the local snapshots of a request.
- If a request is successful, a global snapshot of the request can thus be obtained and the recorded state will indicate termination of the computation,

Formal Description

- Each **process i** maintains a **logical clock** denoted by **x** , initialized to zero at the start of the computation.
- A process increments its **x** by one each time it becomes idle.
- A basic message sent by a process at its **logical time x** is of the form **$B(x)$** .
- A control message that requests processes to take local snapshot issued by process i at its logical time x is of the form **$R(x, i)$** .
- Each process synchronizes its logical clock x loosely with the **logical clocks x 's** on other processes in such a way that it is the maximum of clock values ever received or sent in messages.
- A process also maintains a **variable k** such that when the process is idle, **(x, k) is the maximum of the values (x, k) on all messages $R(x, k)$** ever received or sent by the process.
- **Logical time is compared as follows: $(x, k) > (x', k')$ iff $(x > x')$ or $((x = x') \text{ and } (k > k'))$** , i.e., a tie between x and x' is broken by the process identification numbers k and k' .

Algorithm

The algorithm is defined by the following four rules.

(R1): When process i is active, it may send a basic message to process j at any time by doing send a $B(x)$ to j .

(R2): Upon receiving a $B(x')$, process i does

let $x := x' + 1$;

if (i is idle) $\rightarrow$ go active.

(R3): When process i goes idle, it does

let $x := x + 1$;

let $k := i$;

send message $R(x, k)$ to all other processes;

take a local snapshot for the request by $R(x, k)$.

(R4): Upon receiving message $R(x', k')$, process i does

$[(x', k') > (x, k)) \wedge (i \text{ is idle}) \rightarrow \text{let } (x, k) := (x', k');$

take a local snapshot for the request by $R(x', k')$;

□

$((x', k') \leq (x, k)) \wedge (i \text{ is idle}) \rightarrow \text{do nothing};$

□

$(i \text{ is active}) \rightarrow \text{let } x := \max(x', x).$

The last process to terminate will have the largest clock value. Therefore, every process will take a snapshot for it, however, it will not take a snapshot for any other process.

2. Termination detection by Weight Throwing By ST Huang (1989)

System Model

- A process called **controlling agent** monitors the computation.
- A communication channel exists between each of the processes and the controlling agent and also between every pair of processes.
- Initially, all processes are in the idle state.
- The weight at each process is zero and the weight at the controlling agent is 1.
- The computation starts when the controlling agent sends a basic message to one of the processes.
- A **non-zero weight W ($0 < W \leq 1$)** is assigned to each process in the active state and to each message in transit in the following manner:

Basic Idea

- When a process sends a message, it sends a part of its weight in the message.
- When a process receives a message, it adds the weight received in the message to its weight.
- Thus, the sum of weights on all the processes and on all the messages in transit is always 1.
- When a process becomes passive, it sends its weight to the controlling agent in a control message, which the controlling agent adds to its weight.
- The controlling agent concludes termination if its weight becomes 1.

Notations

- The **weight on the controlling agent** and a process is in general represented by **W** .
- **$B(DW)$** - a basic message **B** sent as a part of the computation, where **DW is the weight** assigned to it.
- **$C(DW)$** - a control message **C** sent from a process to the controlling agent where **DW is the weight** assigned to it.

Algorithm

The algorithm is defined by the following four rules:

- **Rule 1:** The controlling agent or an active process may send a basic message to one of the processes, say P , by splitting its weight W into W_1 and W_2 such **that $W_1 + W_2 = W$, $W_1 > 0$ and $W_2 > 0$** . It then assigns its weight $W := W_1$ and sends a basic message $B(DW := W_2)$ to P .
- **Rule 2:** On the receipt of the message $B(DW)$, process P adds DW to its weight **$W (W := W + DW)$** . If the receiving process is in the idle state, it becomes active.
- **Rule 3:** A process switches from the active state to the idle state at any time by sending a control message $C(DW := W)$ to the controlling agent and making its **weight $W := 0$** .
- **Rule 4:** On the receipt of a message $C(DW)$, the controlling agent adds DW to its weight **$(W := W + DW)$** . **If $W = 1$** , then it concludes that the computation has terminated.

Correctness of Algorithm

Notations

- **A**: set of weights on all active processes
- **B**: set of weights on all basic messages in transit
- **C**: set of weights on all control messages in transit
- **W_c** : weight on the controlling agent.
- Two invariants **I_1** and **I_2** are defined for the algorithm:

- **I_1** : $W_c + \sum_{W \in (A \cup B \cup C)} W = 1$

- **I_2** : $\forall W \in (A \cup B \cup C), W > 0$

Contd...

- **Invariant I_1 states that sum of weights at the controlling process**, at all active processes, on all basic messages in transit, and on all control messages in transit is always equal to 1.
- **Invariant I_2 states that weight at each active process**, on each basic message in transit, and on each control message in transit is non-zero.
- Hence, $W_c = 1$
 - $\Rightarrow \sum_{W \in (A \cup B \cup C)} W = 0$ (by I_1)
 - $\Rightarrow (A \cup B \cup C) = \phi$ (by I_2)
 - $\Rightarrow (A \cup B) = \phi$.
- $(A \cup B) = \phi$ implies the computation has terminated. Therefore, the algorithm never detects a false termination. Further,
 - $(A \cup B) = \phi$
 - $\Rightarrow W_c + \sum_{W \in C} W = 1$ (by I_1)
- Since the message delay is finite, after the computation has terminated, eventually $W_c = 1$.
- Thus, the algorithm detects a termination in finite time.

3. Spanning-Tree-Based Termination Detection Algorithm

- There are **N processes P_i , $0 \leq i \leq N$** , which are modeled as the **nodes i , $0 \leq i \leq N$** , of a **fixed connected undirected graph**.
- The edges of the graph represent the communication channels.
- The algorithm uses a fixed spanning tree of the graph with **process P_0** at its root which is responsible for termination detection.
- **Process P_0** communicates with other processes to determine their states through signals.
- All leaf nodes report to their parents, if they have terminated.
- A parent node will similarly report to its parent when it has completed processing and all of its immediate children have terminated, and so on.
- The root concludes that termination has occurred, if it has terminated and all of its immediate children have also terminated.

Contd...

- Two waves of signals generated one **moving inward** and other **outward** through the spanning tree.
- Initially, a contracting wave of signals, called **tokens**, moves inward from leaves to the root.
- If this token wave reaches the root without discovering that termination has occurred, the root initiates a second outward wave of **repeat** signals.
- As this repeat wave reaches leaves, the token wave gradually forms and starts moving inward again, this sequence of events is repeated until the termination is detected.

A Simple Algorithm

- Initially, each leaf process is given a token.
- Each leaf process, after it has terminated sends its token to its parent.
- When a parent process terminates and after it has received a token from each of its children, it sends a token to its parent.
- This way, each process indicates to its parent process that the subtree below it has become idle.
- In a similar manner, the tokens get propagated to the root.
- The root of the tree concludes that termination has occurred, after it has become idle and has received a token from each of its children.

A Problem with the algorithm

- This simple algorithm fails under some circumstances, when a process after it has sent a token to its parent, receives a message from some other process, which could cause the process to again become active (See **Figure 15.1**).
- Hence the simple algorithm fails since the process that indicated to its parent that it has become idle, is now active because of the message it received from an active process.
- Hence, the root node just because it received a token from a child, can't conclude that all processes in the child's subtree have terminated.

An Example of the Problem

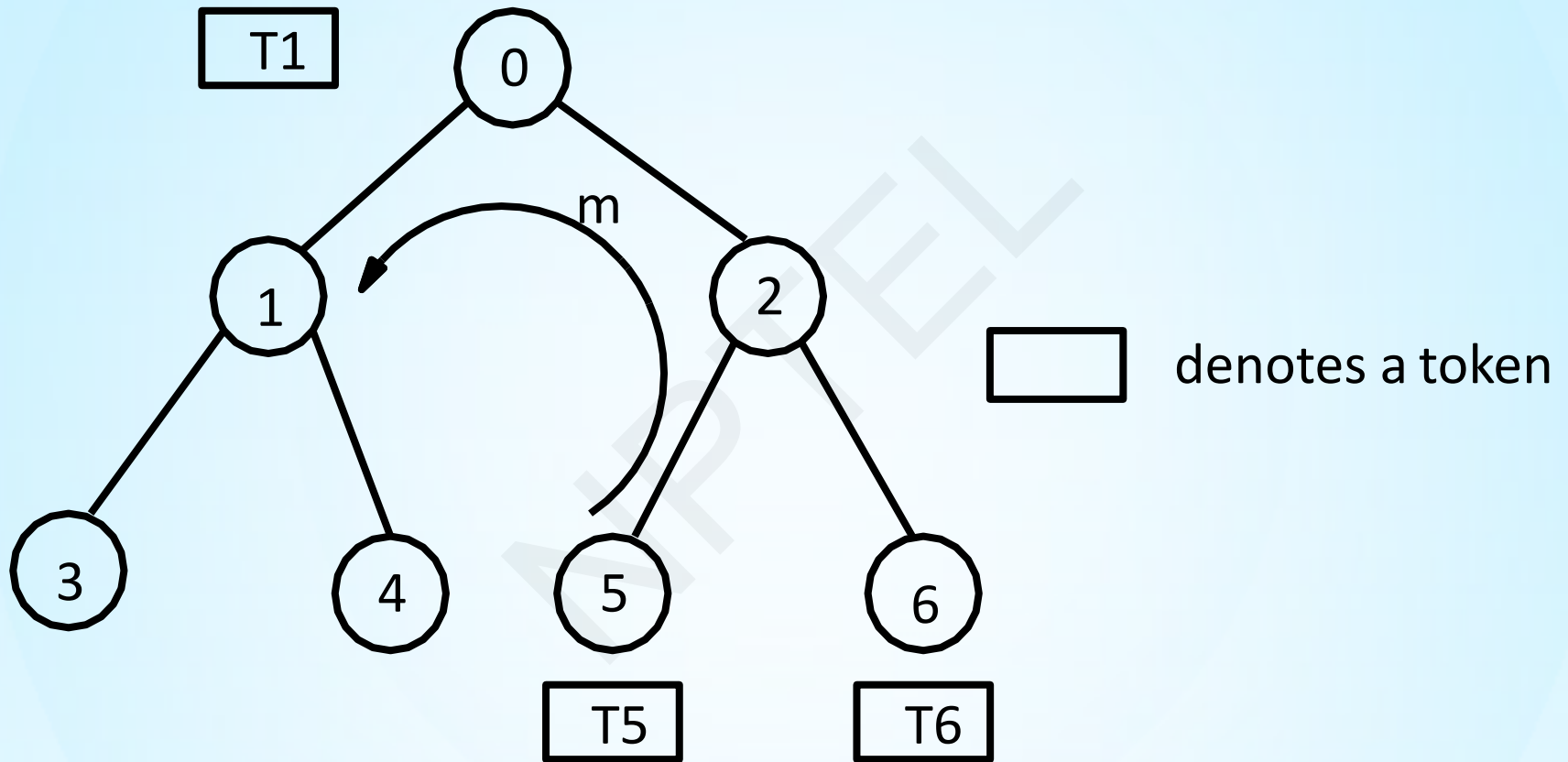


Figure 15.1: An Example of the Problem.

Correct Algorithm Developed By Topor (1984)

- Main idea is to color the processes and tokens and change the color when messages such as in Figure 15.1 are involved.

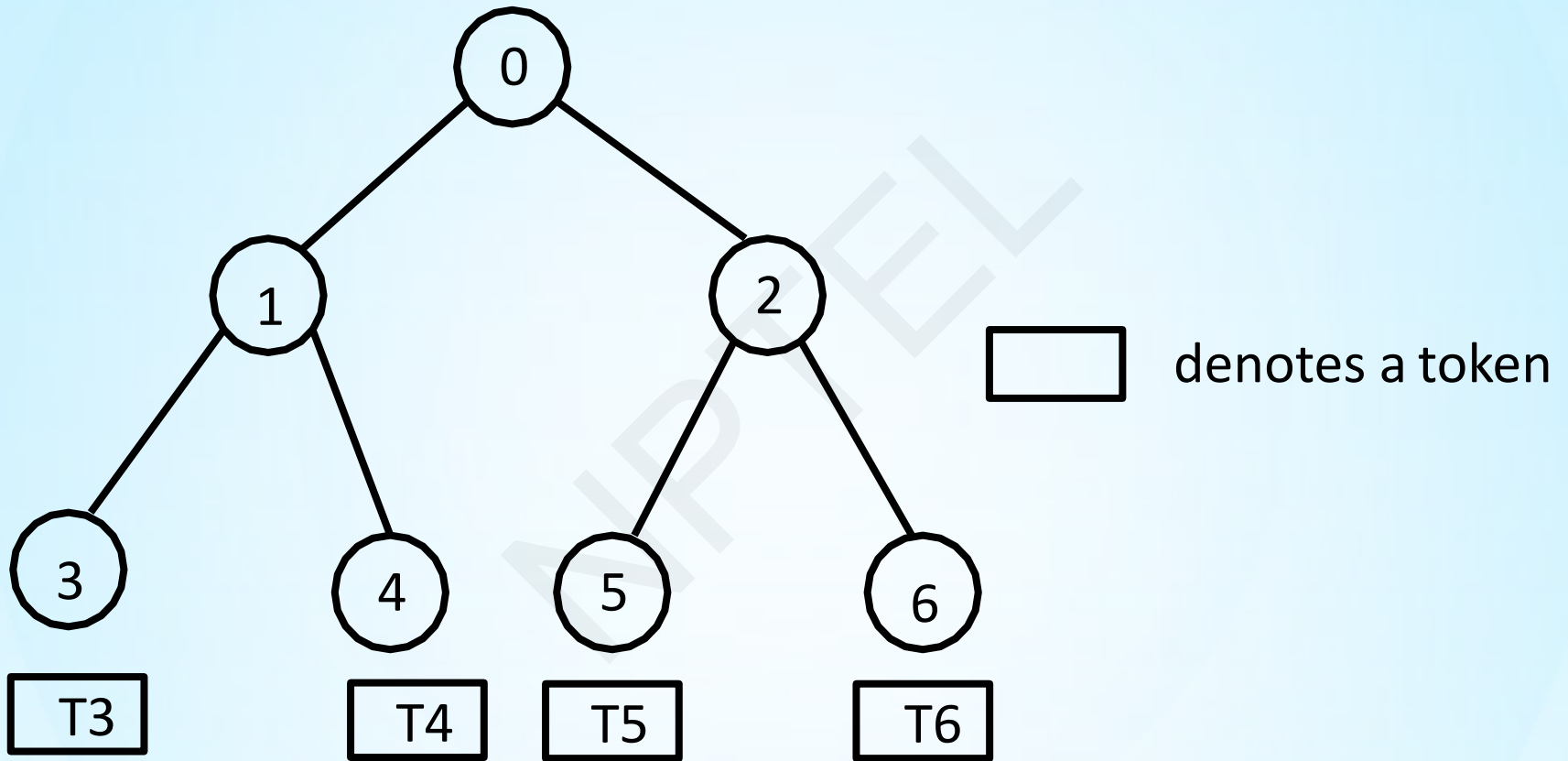
The algorithm works as follows:

1. Initially, each leaf process is provided with a token. The set S is used for book-keeping to know which processes have the token. Hence S will be the set of all leaves in the tree.
2. Initially, all processes and tokens are colored white.
3. When a leaf node terminates, it sends the token it holds to its parent process.
4. A parent process will collect the token sent by each of its children. After it has received a token from all of its children and after it has terminated, the parent process sends a token to its parent.
5. A process turns black when it sends a message to some other process. When a process terminates, if its color is black, it sends a black token to its parent.
6. A black process turns back to white, after it has sent a black token to its parent.

Contd...

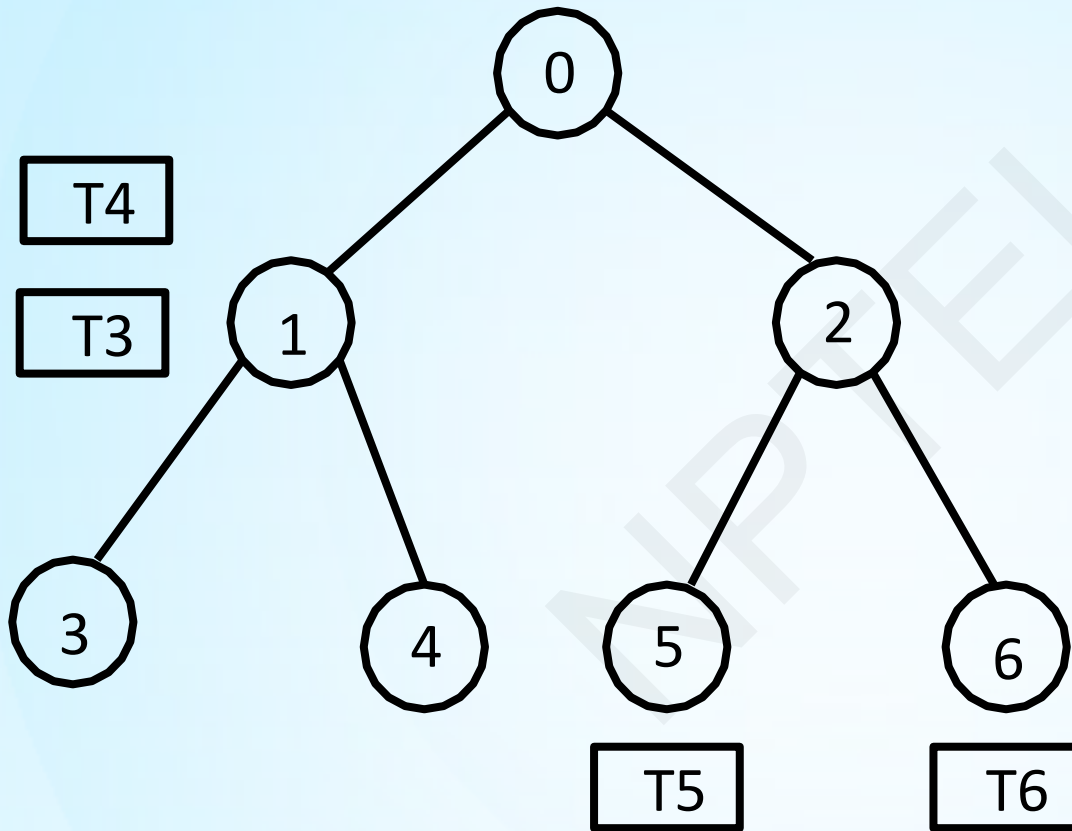
6. A parent process holding a black token (from one of its children), sends only a black token to its parent, to indicate that a message-passing was involved in its subtree.
7. Tokens are propagated to the root in this fashion. The root, upon receiving a black token, will know that a process in the tree had sent a message to some other process. Hence, it restarts the algorithm by sending a Repeat signal to all its children.
8. Each child of the root propagates the Repeat signal to each of its children and so on, until the signal reaches the leaves.
9. The leaf nodes restart the algorithm on receiving the Repeat signal.
10. The root concludes that termination has occurred, if
 1. it is white,
 2. it is idle, and
 3. it received a white token from each of its children.

Example



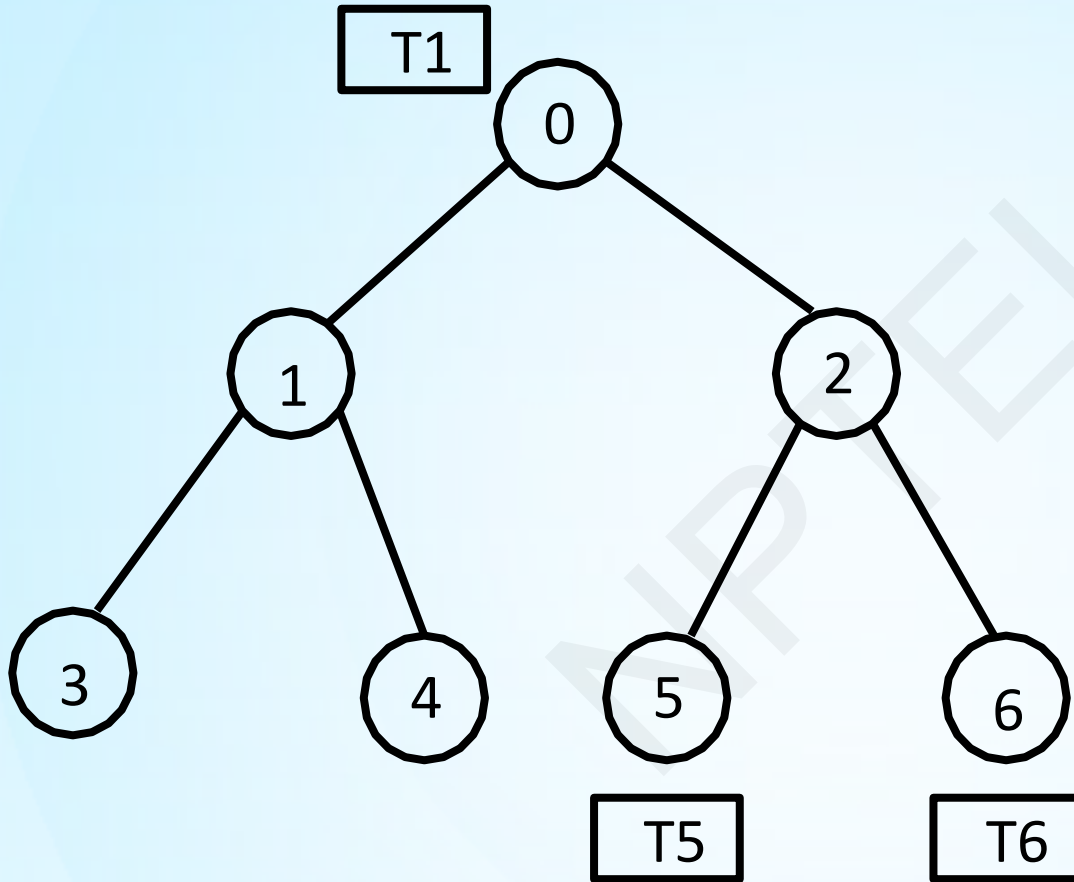
All leaf nodes have tokens. $S = \{3, 4, 5, 6\}$.

Contd...



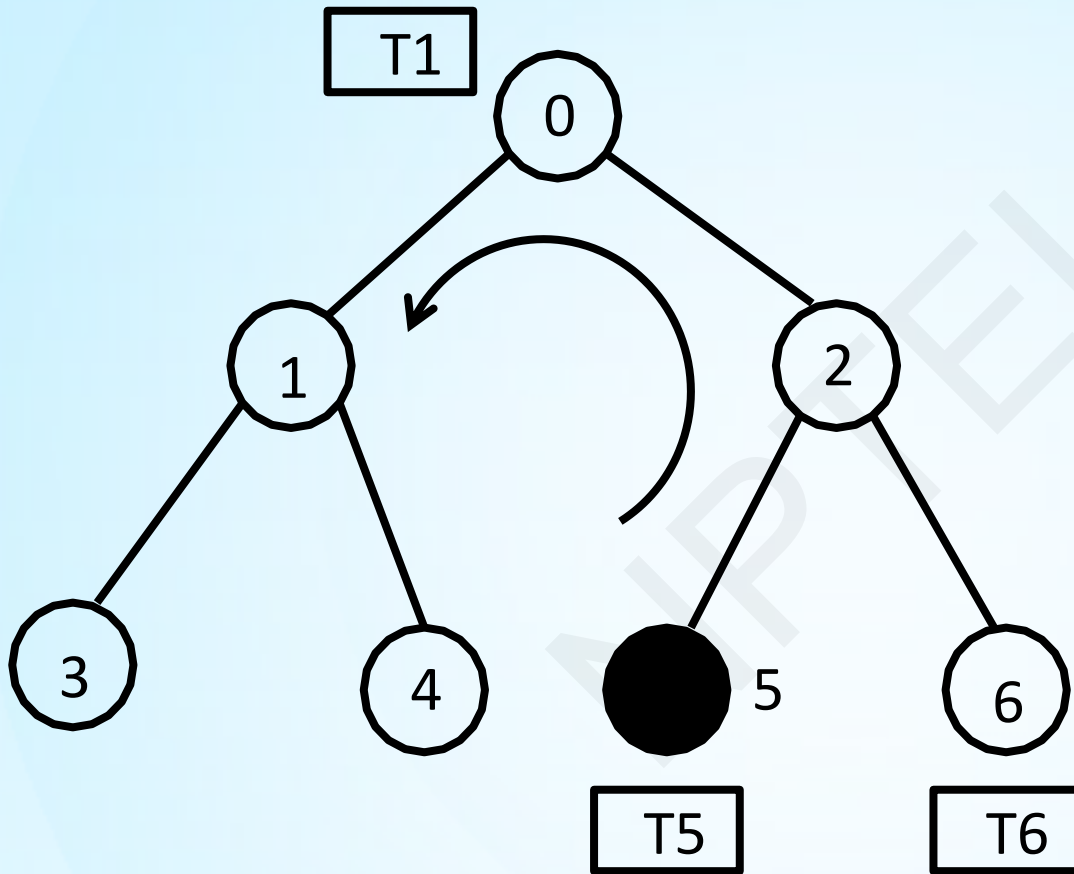
Nodes 3 and 4 become idle. $S = \{1, 5, 6\}$.

Contd...



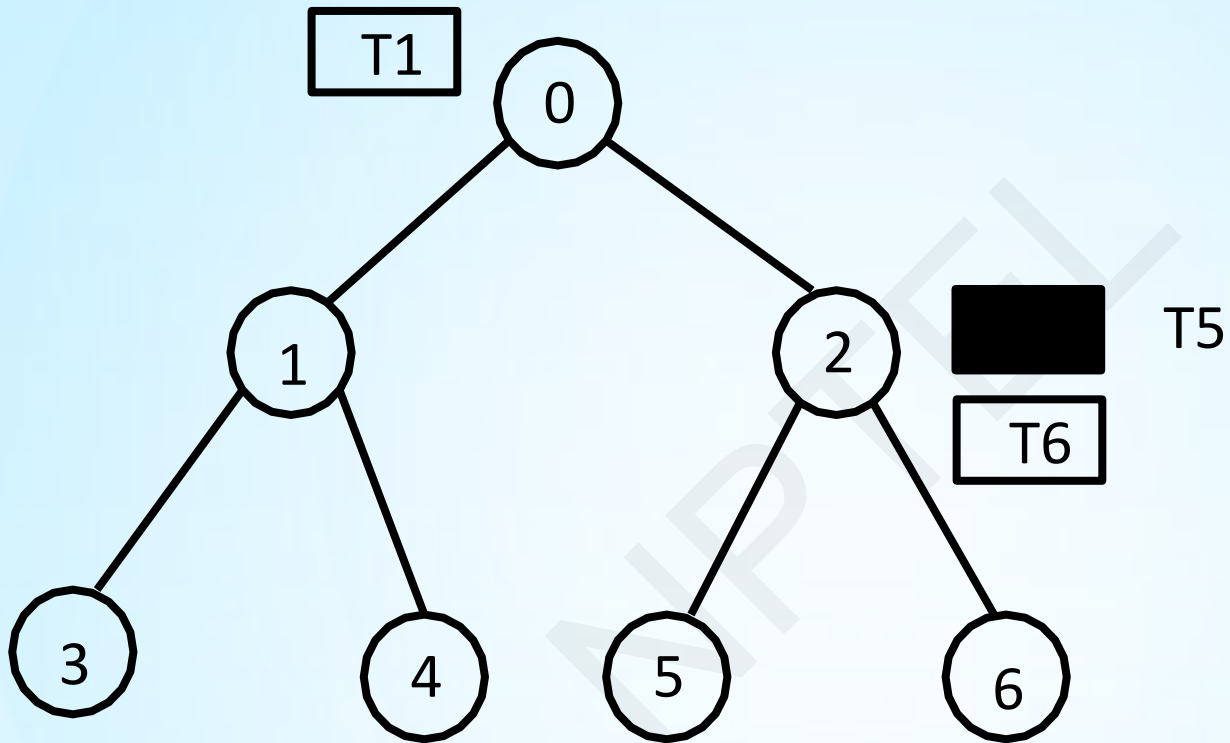
Node 1 becomes idle. $S = \{0, 5, 6\}$.

Contd...



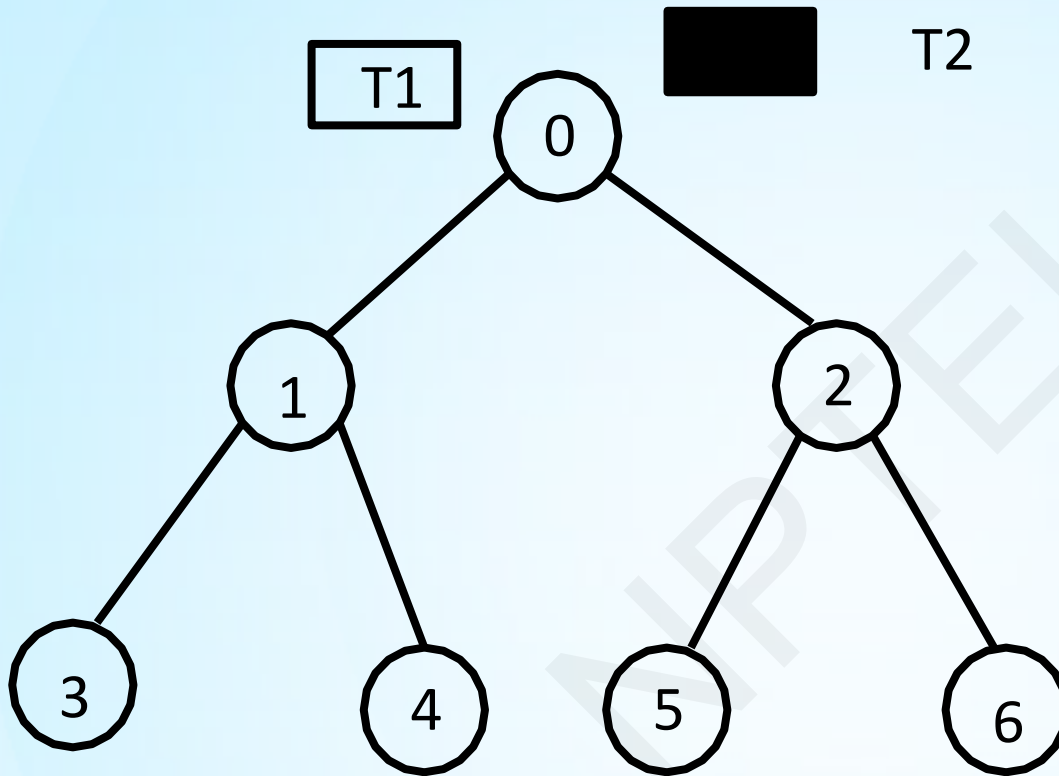
Node 5 sends a message to node 1.

Contd...



Nodes 5 and 6 become idle. $S = \{0, 2\}$.

Contd...



Node 2 becomes idle. $S = 0$. Node 0 initiates a repeat signal.

Performance

- The **best case message complexity** of the algorithm is **$O(N)$** , where N is the number of processes in the computation, which occurs when all nodes send all computation messages in the first round.
- The **worst case complexity** of the algorithm is **$O(N*M)$** , where M is the number of computation messages exchanged, which occurs when only computation message is exchanged every time the algorithm is executed.

Conclusion

- Determining if a distributed computation has terminated is a fundamental problem in distributed systems. **Detection of the termination of a distributed computation is a nontrivial task** since no process has complete knowledge of the global state.
- A number of algorithms have been developed to detect the termination of a distributed computation.
- These algorithms are based on the **concepts of snapshot collection, weight throwing and spanning-tree**, etc.
- In this lecture, we described a set of representative termination detection algorithms.
- In upcoming lecture, we will discuss about '**Message Ordering and Group Communication**'

Lecture 16

Message Ordering and Group Communication



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about '**Termination Detection**' and set of representative termination detection algorithms based on the concepts of **snapshot collection, weight throwing and spanning-tree etc.**

Content of this Lecture:

- In this lecture, we will discuss about:
 - (i) Message ordering
 - (ii) Group communication and
 - (iii) Multicasts

Introduction

- At the core of distributed computing is the communication by message passing among the processes participating in the application.
- In this lecture, we will study **several message ordering paradigms** for communication, such as asynchronous, synchronous, FIFO, causally ordered, and non-FIFO orderings. These orders form a hierarchy. Then we will examine few algorithms to implement these orderings.
- **Group communication** is an important aspect of communication in distributed systems. Causal order and total order are the popular forms of ordering when doing group multicasts and broadcasts. Algorithms to implement these orderings in group will also be discussed.

Notations

- Network (N, L) ; event set $(E, \prec)$
- Message m^i : send and receive events s^i and r^i
- Send and receive events: s and r
- Message M , $send(M)$, and $receive(M)$
- Corresponding events: $a \sim b$ denotes a and b occur at the same process
- Send-receive pairs: $T = \{(s, r) \in E_i \times E_j \mid s \text{ corresponds to } r\}$

Message Ordering Paradigms

Message Ordering Paradigms

- The **order of delivery of messages** in a distributed system is an important aspect of system executions because it determines the messaging behavior that can be expected by the distributed program.
- **Distributed program logic greatly depends on this order of delivery.** To simplify the task of the programmer, programming languages in conjunction with the middleware provide certain well-defined message delivery behavior.
- The programmer can then code the program logic with respect to this behavior. Several orderings on messages have been defined: **(i) non-FIFO, (ii) FIFO, (iii) causal order, and (iv) synchronous order.**

Definitions (1)

- 1. Asynchronous execution (A-execution):** It is an execution $(E, \prec)$ for which the causality relation is a partial order.
- 2. FIFO executions:** A FIFO execution is an A-execution in which, for all (s, r) and $(s', r') \in T$, $(s \sim s' \text{ and } r \sim r' \text{ and } s \prec s') \Rightarrow r \prec r'$
- 3. Causal order (CO):** A CO execution is an A-execution in which, for all (s, r) and $(s', r') \in T$, $(r \sim r' \text{ and } s \prec s') \Rightarrow r \prec r'$
- 4. Causal order (CO) for Implementations:** If $send(m^1) \prec send(m^2)$ then for each common destination d of messages m^1 and m^2 , $deliver_d(m^1) \prec deliver_d(m^2)$ must be satisfied.

Definitions (2)

5. Message Order (MO): A MO execution is and A-execution in which,
for all (s, r) and $(s', r') \in T$, $s \prec s' \Rightarrow \neg(r' \prec r)$

6. Empty-Interval (EI) execution: A execution $(E, \prec)$ is an empty-interval (EI) execution if for each pair of events $(s, r) \in T$, the open interval set $\{x \in E \mid s \prec x \prec r\}$ in the partial order is empty.

7. Synchronous Executions (SYNC): An execution $(E;)$ for which the causality relation is a partial order.

- Handshake between sender and receiver
- Instantaneous communication $\Rightarrow$ modified definition of causality, where $s; r$ are atomic and simultaneous, neither preceding the other.

Hierarchy of execution classes

SYNC, CO, FIFO, and A denote the set of all possible executions ordered by synchronous order, causal order, FIFO order, and non- FIFO order, respectively.

$$\text{SYNC} \subset \text{CO} \subset \text{FIFO} \subset \text{A}$$

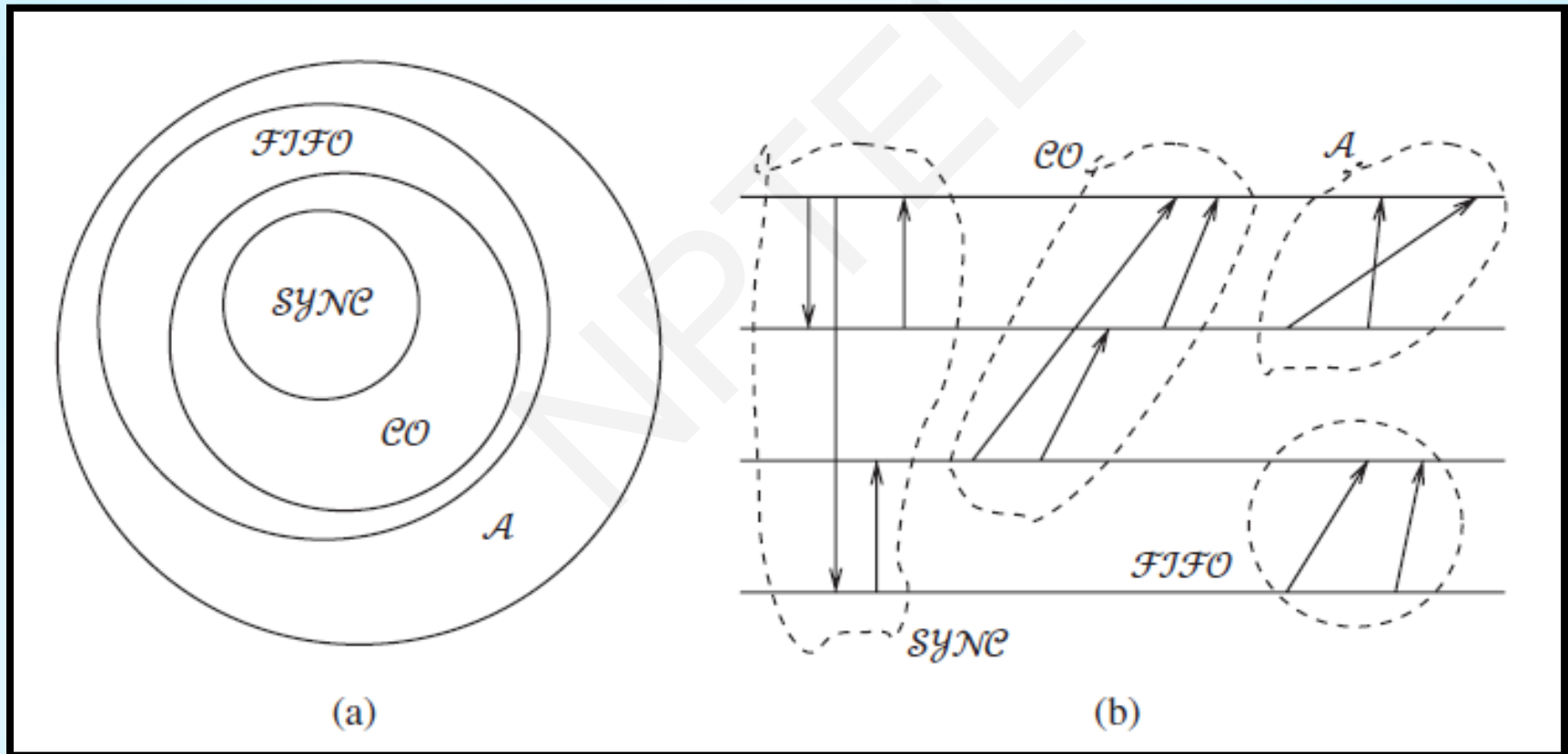


Figure 16.1 Hierarchy of execution classes. (a) Venn diagram. (b) Example executions.

Group Communication

Group Communication

- Processes across a distributed system cooperate **to solve a joint task**. Often, they need to communicate with each other as a group, and therefore there needs to be support **for group communication**.
- A **message broadcast** is the sending of a **message to all members** in the distributed system. The notion of a system can be confined only to those sites/processes participating in the joint application.
- Refining the notion of broadcasting, there is **multicasting** wherein a message is sent to a **certain subset, identified as a group**, of the processes in the system.
- At the other extreme is **unicasting**, which is the familiar **point-to-point** message communication.

Contd...

- **Network layer or hardware-assist multicast** cannot easily provide:
 - Application-specific semantics on message delivery order
 - Adapt groups to dynamic membership
 - Multicast to arbitrary process set at each send
 - Provide multiple fault-tolerance semantics
- Closed group (source part of group) vs. open group (outside of group)
- # groups can be **$O(2^n)$**

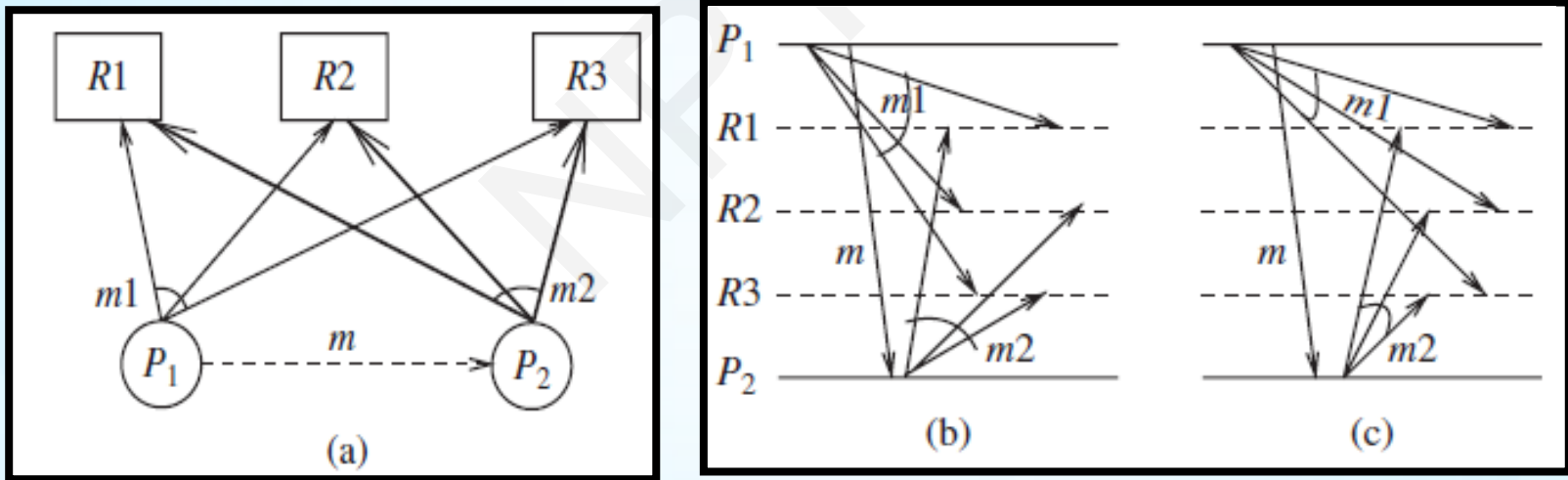


Figure 16.2: (a) Updates to 3 replicas. (b) Causal order (CO) and total order violated. (c) Causal order violated. If m did not exist, (b,c) would not violate CO.

The Raynal–Schiper–Toueg algorithm (1991)

- Intuitively, it seems logical that each **message M** should carry a log of all other messages, or their identifiers, sent causally before **M 's** send event, and sent to the same destination **$\text{dest}(M)$** .
- This log can then be examined to ensure whether it is safe to deliver a message.
- All algorithms aim to reduce this log overhead, and the space and time overhead of maintaining the log information at the processes.
- **Algorithm 16.1** gives a canonical algorithm that is representative of several algorithms that **try to reduce the size of the local space and message space overhead** by various techniques.

Raynal-Schiper-Toueg (RST) Algorithm

```
(local variables)
array of int SENT[1...n, 1...n]
array of int DELIV[1...n]           // DELIV[k] = # messages sent by k that are delivered locally

(1) send event, where  $P_i$  wants to send message  $M$  to  $P_j$ :
(1a) send ( $M, SENT$ ) to  $P_j$ ;
(1b)  $SENT[i, j] \leftarrow SENT[i, j] + 1$ .

(2) message arrival, when ( $M, ST$ ) arrives at  $P_i$  from  $P_j$ :
(2a) deliver  $M$  to  $P_i$  when for each process  $x$ ,
(2b)  $DELIV[x] \geq ST[x, i]$ ;
(2c)  $\forall x, y, SENT[x, y] \leftarrow \max(SENT[x, y], ST[x, y])$ ;
(2d)  $DELIV[j] \leftarrow DELIV[j] + 1$ .
```

Algorithm 16.1 gives a canonical algorithm that is representative of several algorithms that try to reduce the size of the local space and message space overhead by various techniques.

Assumptions/Correctness

- FIFO channels.
- Safety: Step (2a,b).
- Liveness: assuming no failures, finite
- propagation times

Complexity

n^2 ints/ process

n^2 ints/ message

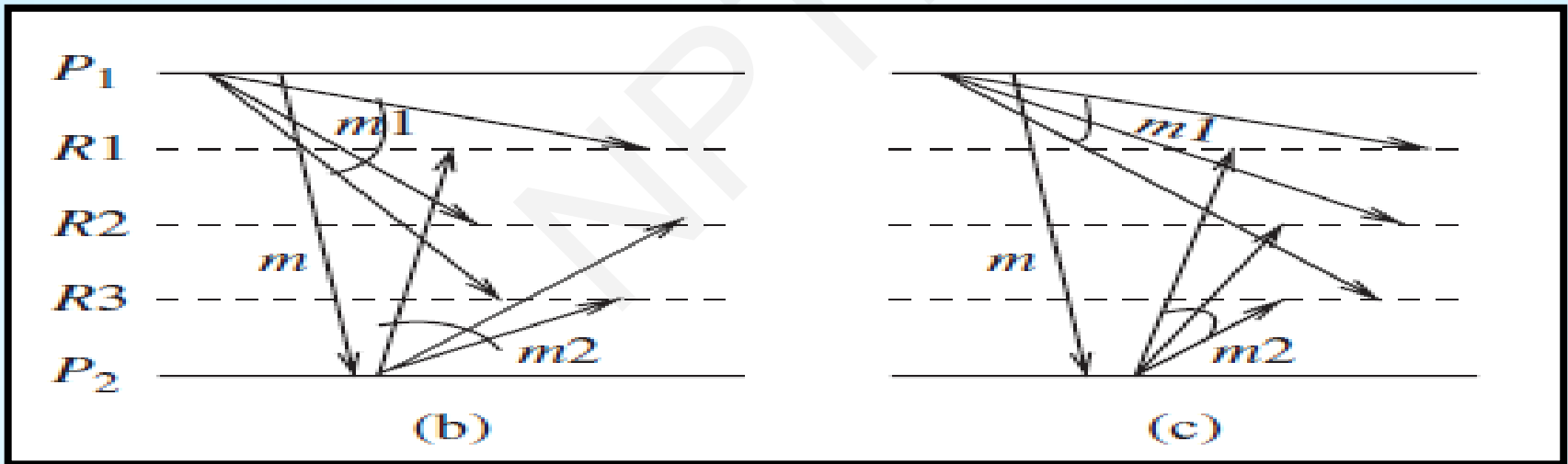
Time per send and receive event: n^2

Total Order

Total Order

Total order, which requires that all messages be received in the same order by the recipients of the messages, is formally defined as follows:

Definition (Total order): For each pair of processes P_i and P_j and for each pair of messages M_x and M_y that are delivered to both the processes, P_i is delivered M_x before M_y if and only if P_j is delivered M_x before M_y .



- Example:** The execution in **Figure 16.2 (b) does not satisfy total order**. Even if the message m did not exist, total order would not be satisfied. The execution in **Figure 16.2(c) satisfies total order**.

Centralized algorithm for Total Order

- (1) When P_i wants to multicast M to group G :
 - (1a) **send** $M(i, G)$ to coordinator.
- (2) When $M(i, G)$ arrives from P_i at coordinator:
 - (2a) **send** $M(i, G)$ to members of G .
- (3) When $M(i, G)$ arrives at P_j from coordinator:
 - (3a) **deliver** $M(i, G)$ to application.

Algorithm 16.2 A centralized algorithm to implement total order and causal order for messages

- Assuming all processes broadcast messages, the centralized solution shown in Algorithm 16.2 enforces total order in a system with FIFO channels.
- Each process sends the message it wants to broadcast to a centralized process, which simply relays all the messages it receives to every other process over FIFO channels.
- It is straightforward to see that total order is satisfied. Furthermore, this algorithm also satisfies causal message order.

Complexity

- **Complexity**

Each message transmission takes two message hops and **exactly n messages** in a system of **n processes**.

- **Drawbacks**

A centralized algorithm has a **single point of failure and congestion**, and is therefore not an elegant solution.

Three-phase distributed algorithm

- A distributed algorithm that enforces total and causal order for closed groups is given in **Algorithm 16.3**. The three phases of the algorithm are first described from the viewpoint of the sender, and then from the viewpoint of the receiver.
- **Sender**
 - **Phase 1** In the first phase, a process multicasts (line 1b) the **message M** with a locally unique tag and the local timestamp to the group members.
 - **Phase 2** In the second phase, the sender process awaits a reply from all the group members who respond with a tentative proposal for a revised timestamp for that message M. The await call in line 1d is **non-blocking**, i.e., any other messages received in the meanwhile are processed. Once all expected replies are received, the process computes the maximum of the proposed timestamps for M, and uses the maximum as the final timestamp.
 - **Phase 3** In the third phase, the process multicasts the **final timestamp** to the group in line (1f).

Contd...

- **Receivers**
- **Phase 1** In the first phase, the receiver receives the message with a tentative/proposed timestamp. It updates the variable priority that tracks the highest proposed timestamp (line 2a), then revises the proposed timestamp to the priority, and places the message with its tag and the revised timestamp at the tail of the queue **temp_Q** (line 2b). In the queue, the entry is marked as undeliverable.
- **Phase 2** In the second phase, the receiver sends the revised timestamp (and the tag) back to the sender (line 2c). The receiver then waits in a non-blocking manner for the final timestamp (correlated by the message tag).

Contd...

- **Phase 3** In the third phase, the final timestamp is received from the multicaster (line 3). The corresponding message entry in **temp_Q** is identified using the tag (line 3a), and is marked as deliverable (line 3b) after the revised timestamp is overwritten by the final timestamp (line 3c). The queue is then resorted using the timestamp field of the entries as the key (line 3c).
- As the queue is already sorted except for the modified entry for the message under consideration, that message entry has to be placed in its sorted position in the queue. If the message entry is at the head of the **temp_Q**, that entry, and all consecutive subsequent entries that are also marked as deliverable, are **dequeued** from **temp_Q**, and enqueued in **deliver_Q** in that order (the loop in lines 3d–3g).

Algorithm 16.3 3-phase Algorithm Code

```

record Q_entry
    M: int;                                     // the application message
    tag: int;                                   // unique message identifier
    sender_id: int;                             // sender of the message
    timestamp: int;                             // tentative timestamp assigned to message
    deliverable: boolean;                       // whether message is ready for delivery

(local variables)
queue of Q_entry: temp_Q, delivery_Q
int: clock                                     // Used as a variant of Lamport's scalar clock
int: priority                                 // Used to track the highest proposed timestamp

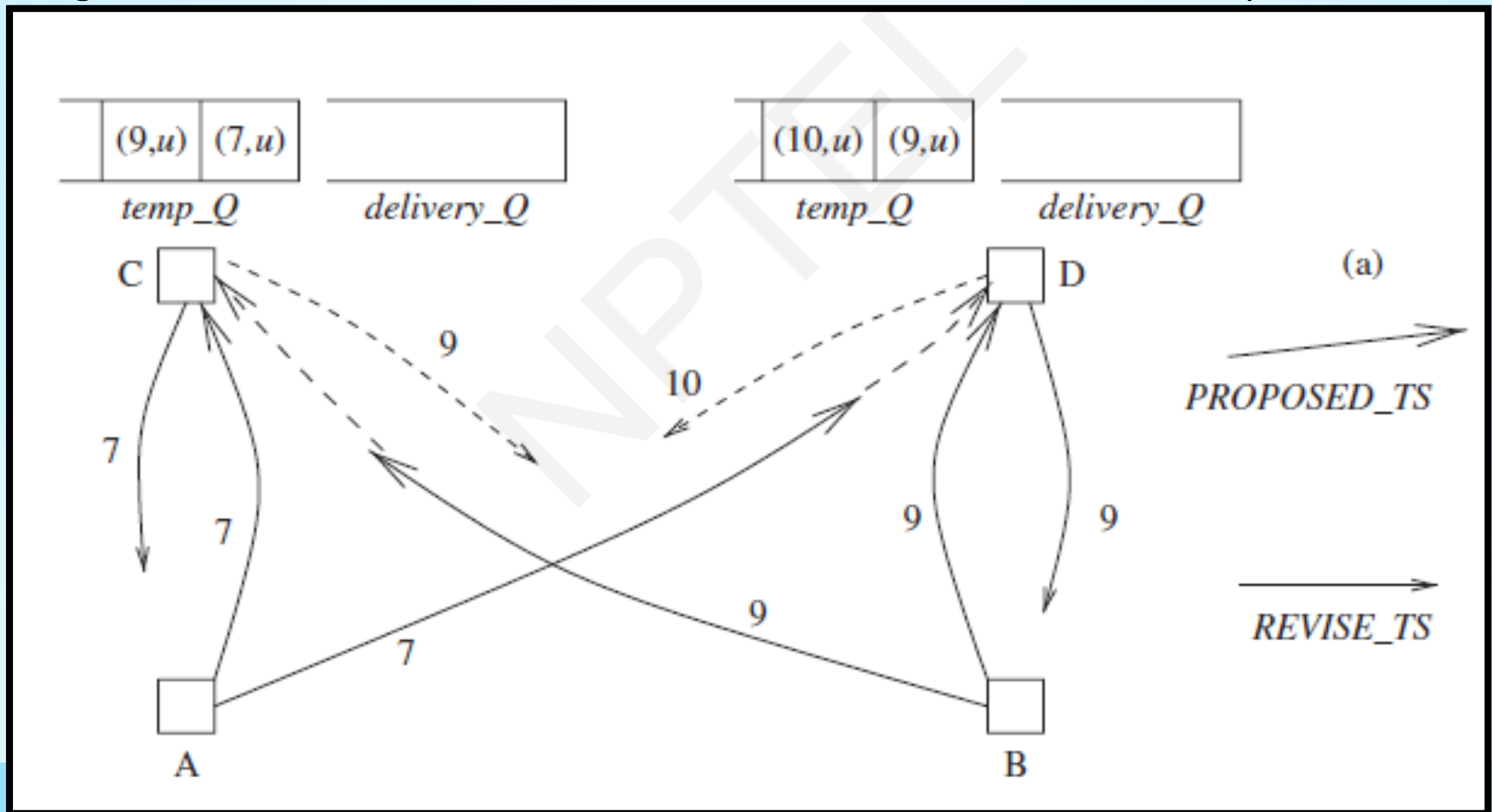
(message types)
REVISE_TS(M, i, tag, ts)                     // Phase 1 message sent by  $P_i$ , with initial timestamp ts
PROPOSED_TS(j, i, tag, ts)                   // Phase 2 message sent by  $P_j$ , with revised timestamp, to  $P_i$ 
FINAL_TS(i, tag, ts)                         // Phase 3 message sent by  $P_i$ , with final timestamp

(1) When process  $P_i$  wants to multicast a message  $M$  with a tag tag:
(1a)  $clock = clock + 1$ ;
(1b) send  $REVISE\_TS(M, i, tag, clock)$  to all processes;
(1c)  $temp\_ts = 0$ ;
(1d) await  $PROPOSED\_TS(j, i, tag, ts_j)$  from each process  $P_j$ ;
(1e)  $\forall j \in N$ , do  $temp\_ts = \max(temp\_ts, ts_j)$ ;
(1f) send  $FINAL\_TS(i, tag, temp\_ts)$  to all processes;
(1g)  $clock = \max(clock, temp\_ts)$ .
(2) When  $REVISE\_TS(M, j, tag, clk)$  arrives from  $P_j$ :
(2a)  $priority = \max(priority + 1, clk)$ ;
(2b) insert  $(M, tag, j, priority, undeliverable)$  in  $temp\_Q$ ; // at end of queue
(2c) send  $PROPOSED\_TS(i, j, tag, priority)$  to  $P_j$ .
(3) When  $FINAL\_TS(j, tag, clk)$  arrives from  $P_j$ :
(3a) Identify entry  $Q\_entry(tag)$  in  $temp\_Q$ , corresponding to tag;
(3b) mark  $q_{tag}$  as deliverable;
(3c) Update  $Q\_entry.timestamp$  to  $clk$  and re-sort  $temp\_Q$  based on the timestamp field;
(3d) if  $head(temp\_Q) = Q\_entry(tag)$  then
(3e)     move  $Q\_entry(tag)$  from  $temp\_Q$  to  $delivery\_Q$ ;
(3f)     while  $head(temp\_Q)$  is deliverable do
(3g)         move  $head(temp\_Q)$  from  $temp\_Q$  to  $delivery\_Q$ .
(4) When  $P_i$  removes a message  $(M, tag, j, ts, deliverable)$  from  $head(delivery\_Q_i)$ :
(4a)  $clock = \max(clock, ts) + 1$ .

```

Example: a) **PROPOSED_TS** and **REVISE_TS** messages.

Example: An example execution to illustrate the algorithm is given in Figure 16.3(a). Here, A and B multicast to a set of destinations and C and D are the common destinations for both multicasts. A snapshot for **PROPOSED_TS** and **REVISE_TS** messages. The dashed lines show the further execution after the snapshot.



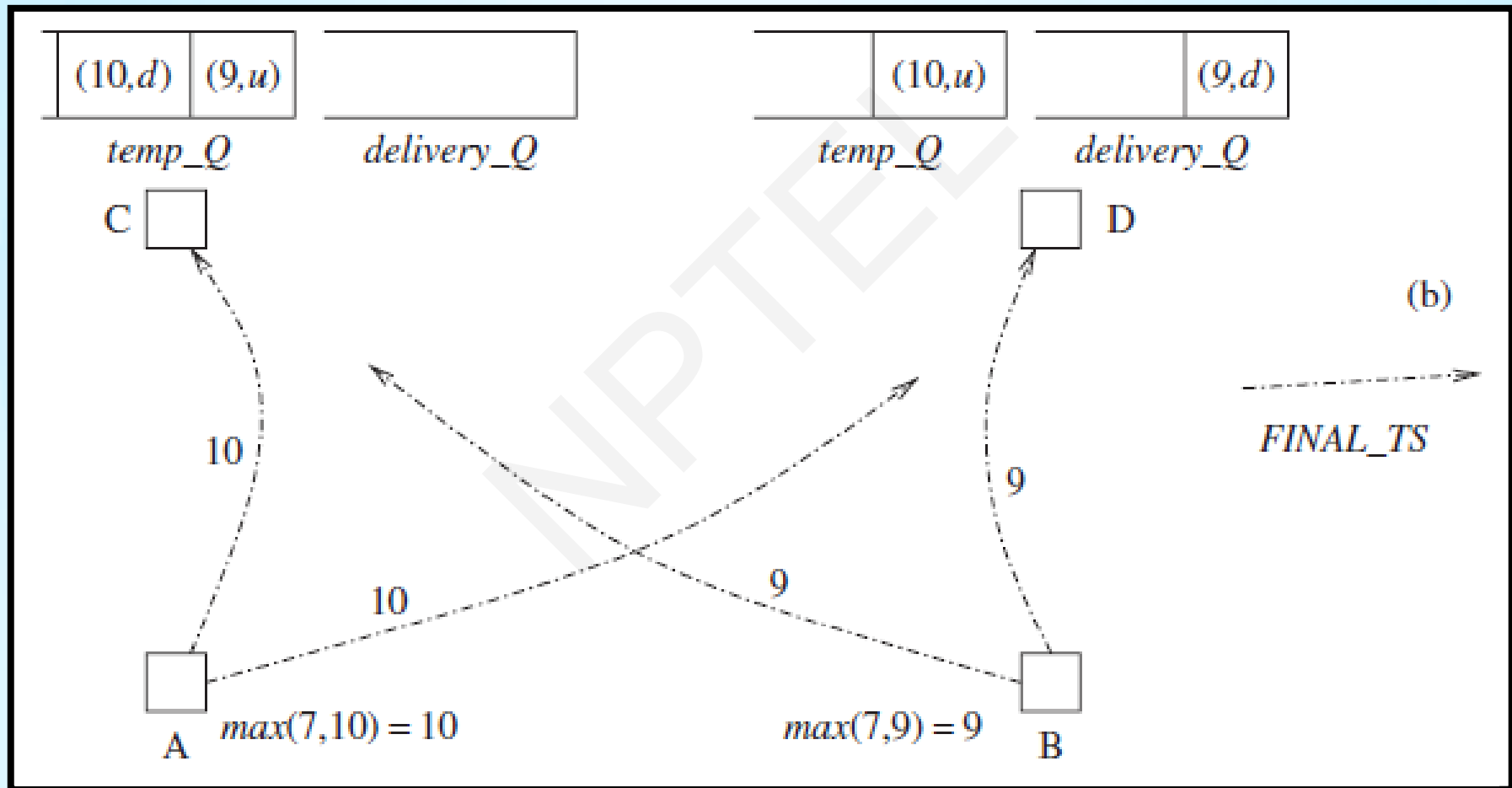
Sequence of Steps

Figure 16.3(a) The main sequence of steps is as follows:

1. A sends a **REVISE_TS(7)** message, having timestamp 7.
B sends a **REVISE_TS(9)** message, having timestamp 9.
2. C receives A's **REVISE_TS(7)**, enters the corresponding message in temp_Q, and marks it as undeliverable; priority = 7.
C then sends **PROPOSED_TS(7)** message to A.
3. D receives B's **REVISE_TS(9)**, enters the corresponding message in temp_Q, and marks it as undeliverable; priority = 9.
D then sends **PROPOSED_TS(9)** message to B.
4. C receives B's **REVISE_TS(9)**, enters the corresponding message in temp_Q, and marks it as undeliverable; priority = 9.
C then sends **PROPOSED_TS(9)** message to B.
5. D receives **A's REVISE_TS(7)**, enters the corresponding message in temp_Q, and marks it as undeliverable; priority = 10. D assigns a tentative timestamp value of 10, which is greater than all of the timestamps on REVISE_TSs seen so far, and then sends **PROPOSED_TS(10)** message to A.

Example: (b) The FINAL_TS messages

Figure 16.3 (b) The FINAL_TS messages in the example.



Sequence of Steps

Figure 16.3(b) The continuing sequence of main steps is as follows:

6. When A receives **PROPOSED_TS(7)** from C and **PROPOSED_TS(10)** from D, it computes the final timestamp as **$\max(7, 10) = 10$** , and sends **FINAL_TS(10)** to C and D.
7. When B receives **PROPOSED_TS(9)** from C and **PROPOSED_TS(9)** from D, it computes the final timestamp as **$\max(9, 9) = 9$** , and sends **FINAL_TS(9)** to C and D.
8. C receives **FINAL_TS(10)** from A, updates the corresponding entry in **temp_Q** with the timestamp, resorts the queue, and marks the message as deliverable. As the message is not at the head of the queue, and some entry ahead of it is still undeliverable, the message is not moved to **delivery_Q**.
9. D receives **FINAL_TS(9)** from B, updates the corresponding entry in **temp_Q** by marking the corresponding message as deliverable, and resorts the queue. As the message is at the head of the queue, it is moved to **delivery_Q**.

Contd...

The following further steps will occur:

10. When C receives **FINAL_TS(9)** from B, it will update the corresponding entry in **temp_Q** by marking the corresponding message as deliverable. As the message is at the head of the queue, it is moved to **the delivery_Q**, and the next message (of A), which is also deliverable, is also moved to the **delivery_Q**.
11. When D receives **FINAL_TS(10)** from A, it will update the corresponding entry in **temp_Q** by marking the corresponding message as deliverable. As the message is at the head of the queue, it is moved to the **delivery_Q**.

Complexity

- This algorithm uses **three phases**
- **$3(n - 1)$** messages for **$n - 1$** destinations
- Delay: **3 message hops**
- Also implements **causal order**

Note:

1. The **3 phase algorithm** is closely structured along the lines of **Lamport's algorithm** for mutual exclusion.
2. **Lamport's mutual exclusion algorithm** has the property that when a process is at the head of its own queue and has received a **REPLY** from all other processes, the **REQUEST** of that process is at the head of all the queues.
3. This can be exploited to deliver the message by all the processes in the same total order (instead of entering the critical section).

A Nomenclature for Multicast

- Four classes of source–destination relationships for open groups:

a) SSSG: Single source and single destination group.

b) MSSG: Multiple sources and single destination group.

c) SSMG: Single source and multiple, possibly overlapping, groups.

d) MSMG: Multiple sources and multiple, possibly overlapping, groups.

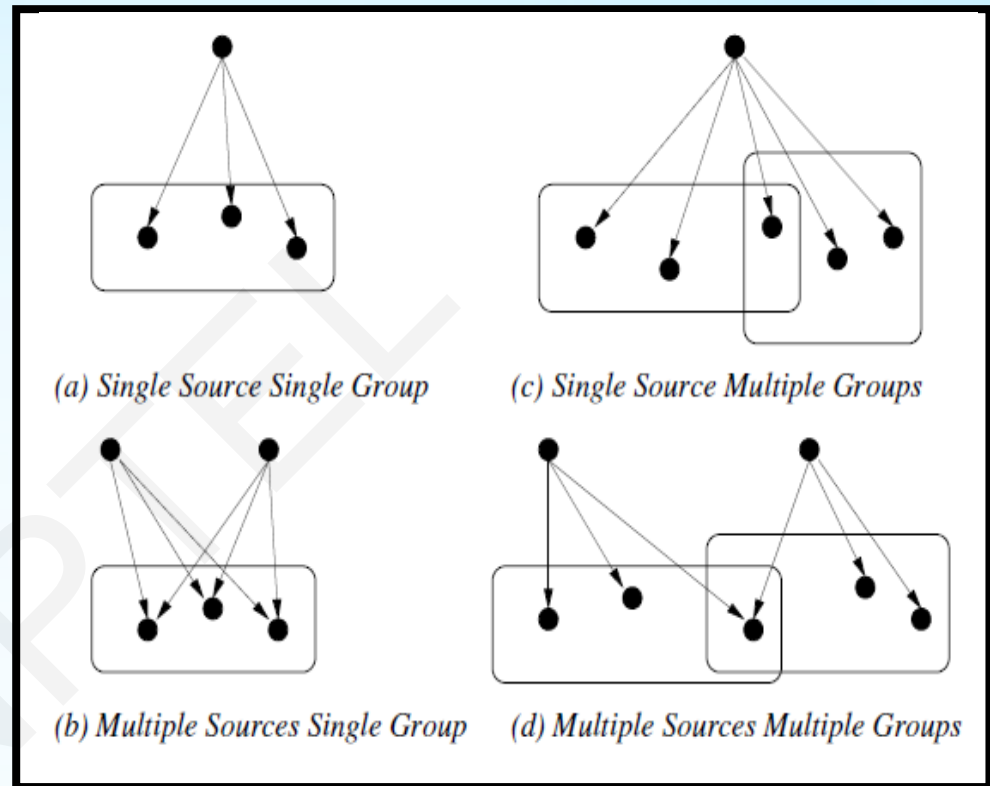


Figure 16.4 : Four classes of source-destinations relationships for open-group multicasts. For closed-group multicasts, the sender needs to be part of the recipient group. SSSG, SSMG: easy to implement MSSG: easy. E.g., Centralized algorithm MSMG: Semi-centralized *propagation tree* approach

Propagation Trees for Multicast: Definitions

- Set of **groups** $G = \{G_1 \dots G_g\}$
- Set of **meta-groups** $MG = \{MG_1, \dots MG_h\}$ with the following properties:
 - Each process belongs to a single meta-group, and has the exact same group membership as every other process in that meta-group.
 - No other process outside that meta-group has that exact group membership.
- MSMG to groups $\rightarrow$ MSSG to meta-groups
- A **distinguished node** in each meta-group acts as its **manager**.
- For each user group G_i , one of its meta-groups is chosen to be its **primary meta-group** (PM), denoted $PM(G_i)$.
- All meta-groups are organized in a **propagation forest/tree** satisfying:
 - For user group G_i , $PM(G_i)$ is at the lowest possible level (i.e., farthest from root) of the tree such that all meta-groups whose destinations contain any nodes of G_i belong to subtree rooted at $PM(G_i)$.

Propagation Trees for Multicast: Example

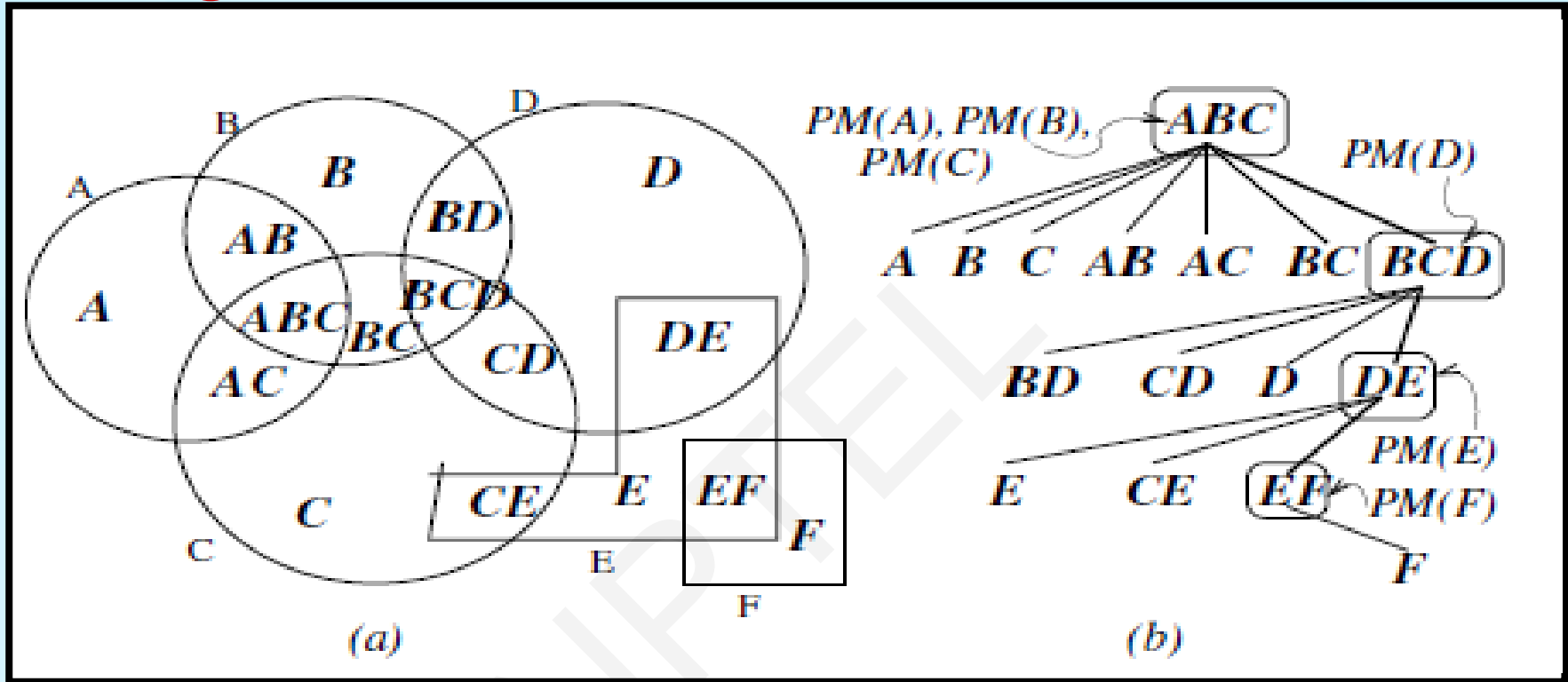


Figure 16.5: Example illustrating a propagation tree. Meta-groups in boldface. (a) Groups A, B, C, D, E and F, and their meta-groups. (b) A *propagation tree*, with the primary meta-groups labeled.

- (ABC) , (AB) , (AC) , and (A) are meta-groups of user group (A).
- (ABC) is $PM(A)$, $PM(B)$, $PM(C)$. (B, C, D) is $PM(D)$. (D, E) is $PM(E)$. (E, F) is $PM(F)$.
- (ABC) is joint with (CD) . Neither subsumes the other and both are a subset of C.
- Meta-group (ABC) is the primary meta-group $PM(A)$, $PM(B)$, $PM(C)$. Meta-group (BCD) is the primary meta-group $PM(D)$. A multicast to D is sent to (BCD) .

Classification of Application-Level Multicast Algorithms

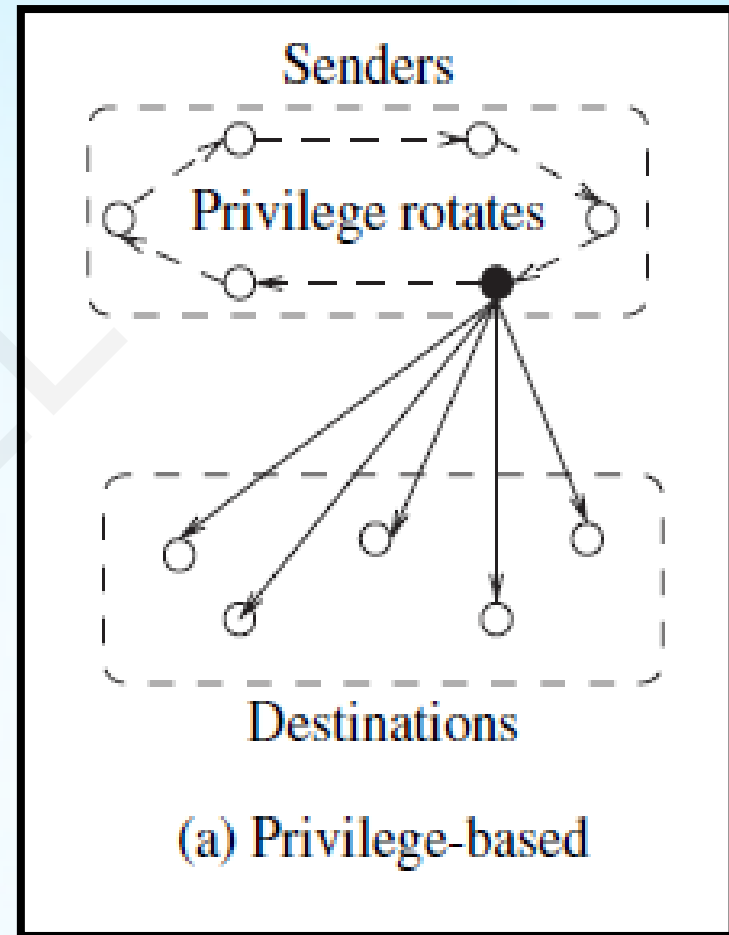
Many multicast protocols have been developed and deployed, but they can all be classified as belonging to one of the following five classes.

1. Communication history-based algorithms:

- Use a part of the **communication history** to guarantee ordering requirements.
- E.g. RST algorithm
- Do not need to track separate groups
- Work for open-group multicasts.

2. Privilege-based:

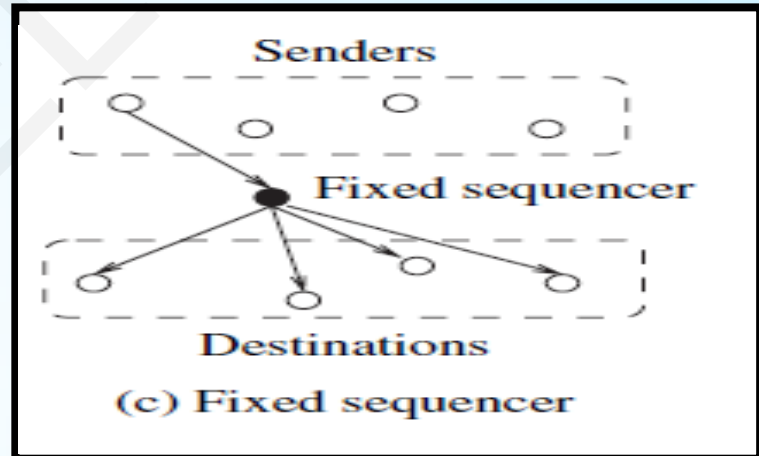
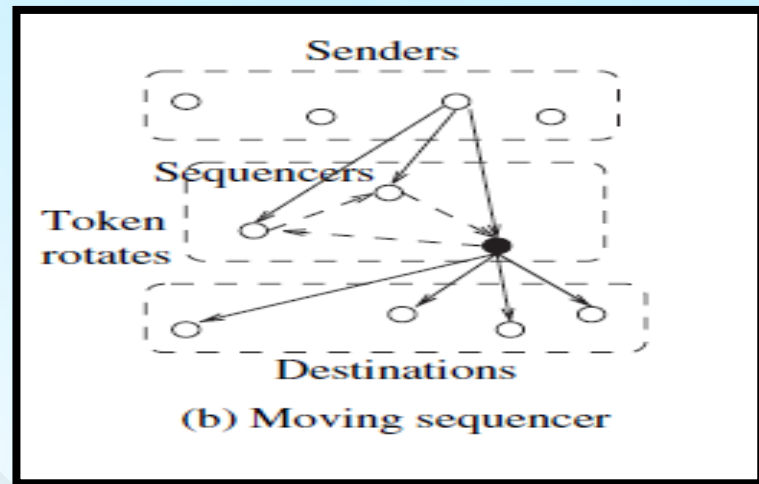
- Token-holder multicasts
- Processes deliver msgs in order of **sequence no.**
- Typically closed groups, and CO & TO.
- E.g., Totem, On-demand.



Contd...

3. Moving sequencer:

- Sequencers' token has **sequence no** and list of messages for which **sequence no** has been assigned (these are sent msgs).
- On receiving token, sequencer assigns **sequence no's** to received but unsequenced messages, and sends the newly sequenced messages to destinations.
- Destinations deliver in order of *sequence no*
- E.g., Chang-Maxemchuk, Pinwheel



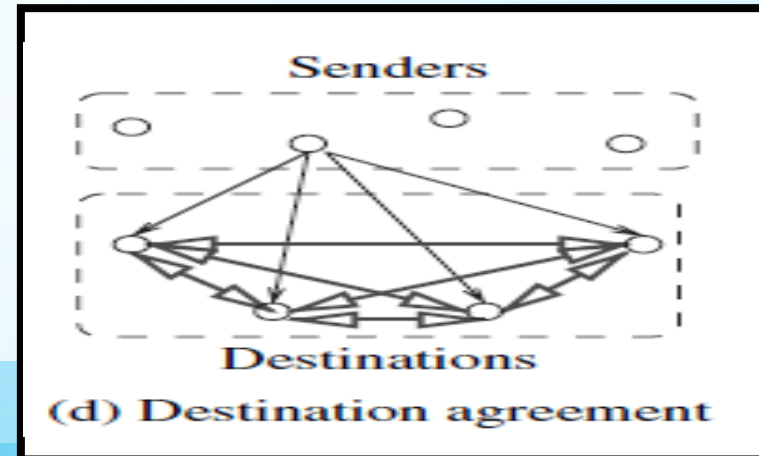
4. Fixed sequencer:

Simplifies moving sequencer approach.

E.g., propagation tree, ISIS, Amoeba, Phoenix

5. Destination agreement:

- Destinations receive limited ordering info.
- Timestamp-based (**Lamport's 3-phase**)
- Agreement-based, among destinations.



Few Other Algorithms for Message Ordering and Group Communication

Author	Algorithm
M. Raynal et al. (1991)	Algorithm for causal order
Kshemkalyani and Singhal (1996,1998)	The space and time optimal algorithm for causal order
K. Birman and T. Joseph (1987)	The algorithm for total order
Garcia-Molina and Spauster (1991) X. Jia (1995), and Chiu and Hsiao (1998)	The algorithm for total order using propagation
Chang and Maxemchuk (1984)	The moving sequencer algorithms
P. Ezhilchelvan et al. (1995)	An efficient fault-tolerant group communication protocol
E. Gilbert and H. Pollack (1968)	Steiner minimal trees
L. Kou et al.	A fast algorithm for Steiner trees
Kompella <i>et al.</i> (1993)	Delay-bounded minimum Steiner
Hadzilacos and Toueg (1993)	Semantics of fault-tolerant group communication
Ballardie <i>et al.</i> (1993)	Core-based trees

Conclusion

- **Inter-process communication via message-passing** is at the core of any distributed system.
- In this lecture, we have discussed **non-FIFO, FIFO, causal order, asynchronous order and synchronous communication paradigms** for ordering messages.
- Then examined several algorithms to implement these orderings.
- **Group communication** is an important aspect of communication in distributed systems.
- **Causal order and total order** are the popular forms of ordering when doing group multicasts and broadcasts. Algorithms to implement these orderings in group communication were also discussed.
- Then we have explained the **propagation trees for multicast** and classification of application-level multicast algorithms.
- In upcoming lecture, we will discuss about '**Self-Stabilization**'.

Lecture 17

Self-stabilization



Rajiv Misra

Dept. of Computer Science &
Engineering

Indian Institute of Technology Patna

rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about **'Message ordering, Group communication and Application level multicast algorithms'**

Content of this Lecture:

- In this lecture, we will discuss the **'concept of self-stabilization', 'related issues in the design of self-stabilizing algorithms and systems'** and **'Dijkstra's self-stabilizing token ring system'**.

Introduction: Concept of Self Stabilization

- The **idea of self-stabilization** in distributed computing was **first proposed by Dijkstra in 1974**.
- **The concept of self-stabilization** is that, regardless of its **initial state**, the system is **guaranteed to converge** to a **legitimate state** in a **bounded amount of time** by itself without any outside intervention. A **non-self-stabilizing system** may never reach a **legitimate state** or it may reach a legitimate state only temporarily.
- The main complication in designing a self-stabilizing distributed system is that nodes **do not have a global memory** that they can access instantaneously. Each node must make decisions based on the **local knowledge** available to it and actions of all nodes must achieve a **global objective**.

Contd...

- **The definition of legitimate and illegitimate states** depends on the particular application. Generally, all illegitimate states are defined to be those states which are not legitimate states.
- **Dijkstra** also gave an example of the concept of self-stabilization using a **self-stabilizing token ring system**. For any given **token ring** when there are **multiple tokens or there is no token**, then such global states are known as **illegitimate states**.
- When we consider a distributed system where a large number of systems are widely distributed and communicate with each other using message passing or shared memory approach, there is a possibility for these systems **to go into an illegitimate state**, for example, if a **message is lost**. The **concept of self-stabilization can help us recover from such situations** in distributed system.

Contd...

- Let us explain the **concept of self-stabilization** using an example. Let us take a group of children and ask them to stand in a circle.
- After few minutes, you will get an almost perfect circle without having to take any further action.
- In addition, you will discover that the shape of this circle is stable, at least until you ask the children to disperse.
- If you force one of the children out of position, the others will move accordingly, moving the entire circle in another position, but keeping its shape unchanged.

Contd...

- In this example, the group of children build a self-stabilizing circle: if some thing goes wrong with the circle, they are able to rebuild the circle by themselves, without any external intervention.
- The **time required for stabilization varies from experiment to experiment**, depending on the (random) initial position. However, if the field size is limited, this time will be bounded.
- The algorithm does not define the position of the circle in the field and so it will not always be the same. The position of each child relative to each other will also vary.

Contd...

- The **self-stabilization principle** applies to any system built on a significant number of components which are evolving independently from one another, but which are cooperating or competing to achieve common goals.
- This applies, in particular, to **large distributed systems** which tend to result from the **integration of many subsystems** and components developed separately at earlier times or by different people.

Contd...

- In this lecture, we will first present the **system model** of a distributed system and present **definitions of self-stabilization**.
- Next, we will discuss **Dijkstra's seminal work** and use it to motivate the topic and then
- We will discuss the **issues arising** from the Dijkstra's original presentation as well as several related issues in the design of self-stabilizing algorithms and systems.

System Model

- The **term distributed system** is used to describe set of computers that communicate over network. Variants of distributed systems have similar fundamental coordination requirements among the communicating entries, whether they are computers, processors or processes.
- Thus an **abstract model** that ignores the specific settings and captures the important characteristics of a distributed system is usually used.
- In a distributed system, each computer runs a program composed of executable statements. Each execution changes the content of the computer's logical memory.
- An **abstract way to model a computer** that executes a program is to use the **state machine model**.

Contd...

- A distributed system model comprises of a set of **n state machines** called processors that communicate with each other.
- We usually denote the ***i*th** processor in the system by **P_i** . Neighbors of a processor are processors that are directly connected to it.
- A processor can directly communicate with its neighbors. A distributed system can be conveniently represented by a graph in which each processor is represented by a node and every pair of neighboring nodes are connected by a link.

Contd...

- The **communication between neighboring processors** can be carried out either by **message passing or shared memory**.
- Communication by writing in and reading from the shared memory usually fits systems with processors that are geographically close together, such as multiprocessor computer.
- A message-passing distributed model fits both processors that are located close to each other as well as that are widely distributed over a network.
- In the message-passing model, neighbors communicate by sending and receiving messages.
- Message passing communication model: **queue(s) Q_{ij} , for messages from P_i to P_j**

Contd...

- It is convenient to identify the state of a computer or a distributed system at a given time, so that no additional information about the **past of the computation** is needed in order to **predict the future behavior** (state transitions) of the computer or the distributed system.
- A full description of a message passing distributed system at a particular time consists of the state of every processor and the content of every queue (messages travelling in the communication links).

Contd...

- The term system configuration (or configuration) is used for such a description.

A configuration is denoted by

$c = (s_1, s_2, \dots, s_n, q_{1,2}, q_{1,3}, \dots, q_{i,j}, \dots, q_{n,n-1}),$

where $s_i, 1 \leq i \leq n$ is the state of P_i and $q_{i,j}$,

$i \neq j$ is the state of queue $Q_{i,j}$, that is, messages sent by P_i to P_j but not yet received.

- The behavior of a system consists of a set of states, a transition relation between those states, and a set of fairness criteria on the transition relation.

Contd...

- The system is usually modeled as a **graph of processing elements** (modeled as state machines), where edges between these elements model unidirectional or bidirectional communication links.
- Let **N be an upper bound on n (the number of nodes in the system)**. The communication network is usually restricted to the neighbors of a particular node.
- Let **$\bar{g}$ denote the diameter of the network** (i.e., the length of the longest unique path between two nodes) and **let Δ denote the upper bound on $\bar{g}$.**

Contd...

- A **network is static** if the communication topology remains fixed. It is dynamic if links and network nodes can go down and recover later.
- In the **context of dynamic systems**, self-stabilization refers to the time after the “final” link or node failure. The term “final failure” is typical in the literature on self-stabilization.
- Since stabilization is only guaranteed eventually, the assumption that faults eventually stop to occur implies that there are no faults in the system for “sufficiently long period” for the system to stabilize.
- In any case, it is assumed that the topology remains connected, i.e., there exists a path between any two nodes.

Contd...

- In the **shared memory model**, processors communicate using shared communication registers (hereafter, called registers). Processors may write in a set of registers and may read from a possibly different set of registers.
- Two neighboring nodes have **access to a common data structure, variable or register** which can store a certain amount of information. These variables can be distinguished between input and output variables (depending on which process can modify them).
- When executing a step, a process may read all its input variables, perform a state transition and write all its output variables in a single atomic operation. This is called **composite atomicity**. A weaker notion of a step (**called read/write atomicity**) also exists where a process can only either read or write its communication variables in one atomic step.

Contd...

- The configuration of a system with **n processors and m communication registers is denoted by**

$$c = (s_1, s_2, s_3 \dots s_n, r_1, r_2 \dots r_m),$$

where $s_i, 1 \leq i \leq n$, is the state of P_i and

$r_j, 1 \leq j \leq m$, is the contents of a communication register.

Contd...

- Algorithms are **modeled as state machines performing a sequence of steps**. A step consists of reading input and the local state, then performing a state transition and writing output.
- Communication can be by exchanging messages over the communication channels. An algorithm may be randomized, i.e., have access to a source of randomness (a random number generator or a random coin flip). If an algorithm is not randomized, we will call it deterministic.
- A related characteristic of a system model is its execution semantics. In self-stabilization, this has been encapsulated within the notion of a **scheduler or daemon** (also demon).
- Under **a central daemon, at most one processing element is allowed** to take a step at the same time.

Definition of Self-stabilization

- We have seen an informal definition of self-stabilization at the beginning.
- **Formally, we define self-stabilization** for a **system S** with respect to a **predicate P** over its set of global states, where P is intended to identify its correct execution. States satisfying P are called **legitimate states** and those not satisfying P are called illegitimate states. We use the terms safe and unsafe interchangeably with legitimate and illegitimate, respectively. A system S is self-stabilizing with respect to predicate P if it satisfies the following two properties:
 - (i) **Closure:** P is closed under the execution of S. That is, once P is established in S, it cannot be falsified.
 - (ii) **Convergence:** Starting from an arbitrary global state, S is guaranteed to reach a global state satisfying P within a finite number of state transitions.

Contd...

- **Arora and Gouda** introduced a more generalized definition of self-stabilization, called **stabilization**, which is defined as follows.
- We define stabilization for a system S with respect to two predicates P and Q , over its set of global states. Predicate Q denotes a restricted start condition. **S satisfies $Q \rightarrow P$ (read as Q stabilizes to P)** if it satisfies the following two properties:
 - (i) **Closure** P is closed under the execution of S . That is, once P is established in S , it cannot be falsified.
 - (ii) **Convergence** If S starts from any global state that satisfies Q , then S is guaranteed to reach a global state satisfying P within a finite number of state transitions.
- Note that **self-stabilization is a special case of stabilization** where Q is always true, that is, if S is self-stabilizing with respect to P , then this may be restated as **$TRUE \rightarrow P$ in S .**

Contd...

Reachable Set:

- Often when a programmer writes a program, he/she does not have a particular definition of safe and unsafe states in mind but develops the program to function from a particular set of start states.
- In such situations, it is reasonable to define as **safe those states that are reachable under normal program execution from the set of legitimate start states**. These states are referred to as the reachable set.
- So, when we say that a program is self-stabilizing without mentioning a predicate, we mean with respect to **the reachable set**. By definition, **the reachable set is closed under program execution**, and it corresponds to a **predicate over the set of states**.

Contd...

Transient failure:

- A **transient failure is temporary (short lived)** and it does not persist.
- A transient failure may be caused by **corruption of local state of processes** or by **corruption of channels or shared memory**.
- A transient failure **may change the state of the system**, but not its behavior.

Issues in the design of self-stabilization algorithms

- A distributed system **comprises of many individual units** and many issue arise in the design of self-stabilization algorithms in distributed system. Some of the **main issues** are as follows:
- Number of states in each of the individual units in a distributed system.
- Uniform and non-uniform algorithms in distributed systems.
- Central and distributed demon.
- Reducing the number of states in a token ring.
- Shared memory models.
- Mutual exclusion.
- Costs of self-stabilization.

Dijkstra's self-stabilizing token ring system

- The mentioned issues can be explained with the help of **Dijkstra's landmark self-stabilizing token ring system.**
- His system consisted of a **set of n finite state machines connected in the form a ring.** He defines a privilege of a machine to be the ability to change its current state.
- This ability is based on a Boolean predicate that consists of its current state and the states of its neighbors. When a machine has a privilege, it is able to change its current state, which is referred to as a move.
- Furthermore, when multiple machines enjoy a privilege at the same time, the choice of the machine that is entitled to make a move is made by a central demon, which arbitrarily decides which privileged machine will make the next move.

Contd...

A legitimate state must satisfy the following constraints:

1. There must be at least one privilege in the system (**liveness or no deadlock**).
2. Every move from a legal state must again put the system into a legal state (**closure**).
3. During an infinite execution, each machine should enjoy a privilege an infinite number of times (**no starvation**).
4. Given any two legal states, there is a series of moves that change one legal state to the other (**reachability**).

Contd...

- **Dijkstra** considered a **legitimate (or legal) state** as one in which exactly one machine enjoys the privilege.
- This corresponds to a **form of mutual exclusion**, because the **privileged process** is the only process that is allowed in its critical section.
- Once the process leaves the critical section, **it passes the privilege to one of its neighbors.**

The number of states in each of the individual units

- The number of states that each machine must have for the self-stabilization is an important issue.
- **Dijkstra offered three solutions for a directed ring with n machines**, $0, 1, \dots, n-1$, each having **K states**, (i) $K \geq n$, (ii) $K = 4$, (iii) $K = 3$.
- It was later proven by **Ghosh** that a **minimum of three states is required in a self-stabilizing ring**.
- In all three algorithms, **Dijkstra** assumed the existence of **at least one exceptional machine** that behaved differently from the others.

First solution ($K \geq n$)

- For any machine, we use the symbols **S**, **L**, and **R** to denote its own state, the state of the left neighbor and the state of the right neighbor on the ring, respectively.

The exceptional machine:

If $L = S$ then

$S := (S+1) \bmod K$

End If;

The other machines:

If $L \neq S$ then

$S := L$

End If;

Contd...

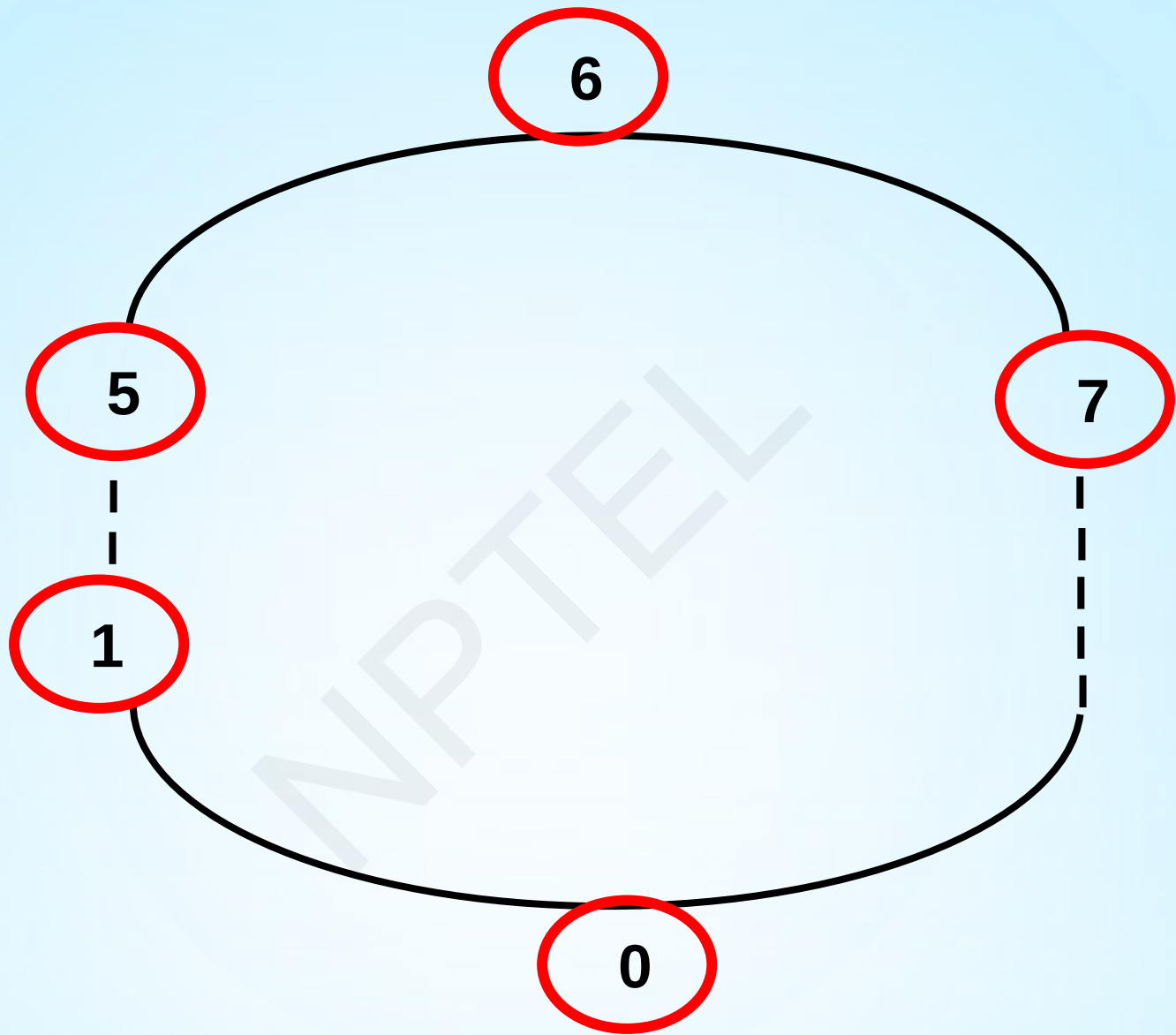
- In this algorithm, except the **exceptional machine(machine 0)**, all other machines follow the same algorithm. **In the ring topology, each machine compares its state with the state of the anti-clockwise neighbor** and if they are not same, it updates its state to be the same as that of its anti-clockwise neighbor.
- So, if there are **n machines** and each of them is initially at a **random state $r \in K$** , then all the machines (except the exceptional machine, **machine 0**) whose states are not the same as their anti-clockwise neighbor are said to be privileged and there is a central demon that decides which of these privileged machines will make the move.

Contd...

- Suppose **machine 6** (assume $n \gg 6$) makes the first move. It is obvious that its state is not the same as that of **machine 5** and hence it had the privilege to make the move and finally sets its state to be the same as that of **machine 5**. Now **machine 6** loses its privilege as its state is same as that of its anti-clockwise neighbor (**machine 5**).
- Next, suppose **machine 7**, whose state is different from the state of **machine 6**, is given the privilege. It results in making the state of **machine 7** the same as that of **machine 6**. Now **machines 5, 6, and 7** are in the same state.
- Eventually, all the machines will be in the same state in the similar manner. At this point, only the exceptional machine (**machine 0**) will be privileged as its condition $L = S$ is satisfied, i.e., its state is the same as that of its anti-clockwise neighbor.

Contd...

- Now there exists only one privilege or token in the system (at **machine 0**). **Machine 0** makes a move and changes its state **from S to $(S+1) \bmod K$** . This will make the next machine, **machine 1**, privileged as its state is not the same as its anti-clockwise neighbor, i.e., **machine 0**. Thus, it can be interpreted as the token is currently with **machine 1**.
- **Machine 1**, as per the algorithm, changes its state to the same state as that of **machine 0**. This will move the token to **machine 2** as its state is now not same as that of **machine 1**. Likewise, the token keeps circulating around the ring and the system is stable.
- This is a simple algorithm, but it requires a **number of states**, which **depends on the size of the ring**, which may be awkward for some applications.



Second solution ($K = 3$)

- The second solution uses only **three-state machines** and is presented in **Algorithm 17.1**
- The state of each machine is in $\{0, 1, 2\}$.
- In the first algorithm, there is only one exceptional machine, machine 0.
- In the second solution, there are two such machines, **machine 0**, referred to as the bottom machine, and **machine $n-1$** , referred to as the top machine.

Contd...

The bottom machine, machine 0:

If $(S+1) \bmod 3 = R$ then
 $S := (S-1) \bmod 3$

The top machine, machine $n-1$:

If $L = R$ and $(L+1) \bmod 3 \neq S$ then
 $S := (L+1) \bmod 3$

The other machines:

If $(S+1) \bmod 3 = L$ then
 $S := L$
If $(S+1) \bmod 3 = R$ then
 $S := R$

Contd...

- In this algorithm, the bottom machine, machine 0, behaves as follows:

If $(S+1) \bmod 3 = R$ then

$S := S-1 \bmod 3$

- Thus, the state of the bottom machine depends upon its current state and the state of its right neighbor.
- The condition $(s+1) \bmod 3$ covers the three possible states; for $s = 0, 1, 2$, we have $(s+1) \bmod 3 = 1, 2, 0$. These result in the following three possibilities:
 1. If $s = 0$ and $r = 1$, then the state of s is changed to 2.
 2. If $s = 1$ and $r = 2$, then the state of s is changed to 0.
 3. If $s = 2$ and $r = 0$, then the state of s is changed to 1.

Contd...

- The top machine, **machine n-1**, behaves as follows:

If $L = R$ and $(L+1) \bmod 3 \neq S$ then

$S := (L+1) \bmod 3$

- The state of the top machine depends upon both its left and right neighbors (the bottom machine). The condition specifies that the **left neighbor (L)** and the **right neighbor (R)** should be in the same state and **$(L+1) \bmod 3$** should not be equal to S . (Note that **$(L+1) \bmod 3$ is 1, 2, 0 when L is 0, 1, 2, respectively**) Thus, the state of the top machine is as follows:
 1. **1**, when its left neighbor is 0.
 2. **2**, when its left neighbor is 1.
 3. **0** when its left neighbor is 2.

Contd...

- All other machines behave as follows:

If $(S+1) \bmod 3 = L$ then

$S := L$

If $(S+1) \bmod 3 = R$ then

$S := R$

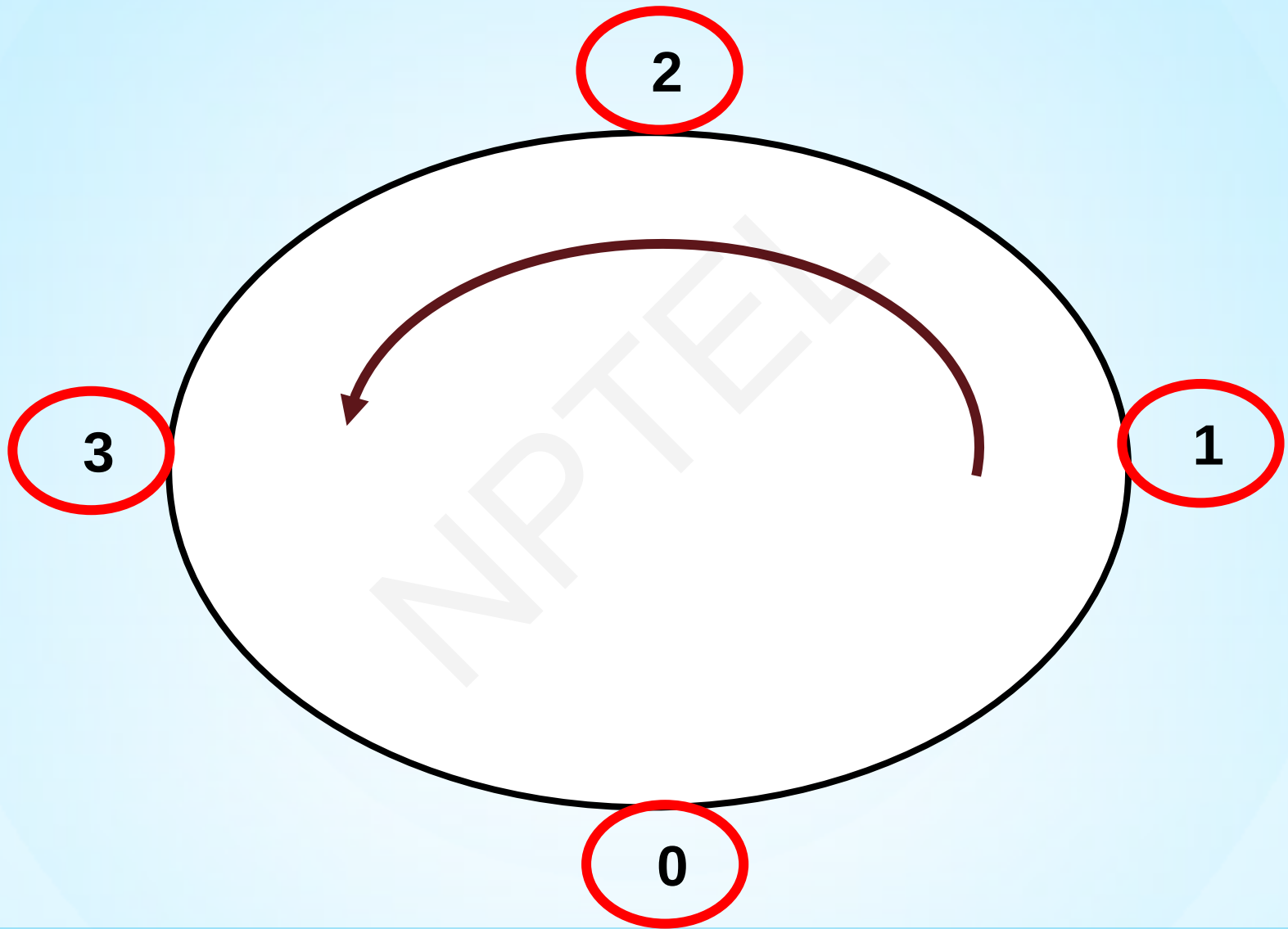
- While finding out the state of the other machines (**machines 1 and 2** in the example below), we first compare the state of a machine with its left neighbor:
 1. **If $s = 0$ and $L = 1$** , then $s = 0$.
 2. **If $s = 1$ and $L = 2$** , then $s = 2$.
 3. **If $s = 2$ and $L = 0$** , then $s = 1$.

Contd...

- If the given conditions are not satisfied, then the machine compares its state with its right neighbor.
- A sample execution of **Dijkstra's three-state algorithm for a ring of four processes (0, 1, 2, 3)** is shown in **Table 17.1. Machine 0 is the bottom machine and machine 3 is the top machine.**
- The last column in the table gives the number of the machine chosen to make the next move.
- Initially, three privileges exist in the system. The number of privileges decreases until only one privilege is left in the system.

Table 17.1 An example execution of Dijkstra's three-state algorithm

State of machine 0	State of machine 1	State of machine 2	State of machine 3	Privileged machines	Machine to make move
0	1	0	2	0, 2, 3	0
2	1	0	2	1, 2	1
2	2	0	2	1	1
2	0	0	2	0	0
1	0	0	2	1	1
1	1	0	2	2	2
1	1	1	2	2	2
1	1	2	2	1	1
1	2	2	2	0	0
0	2	2	2	1	1
0	0	2	2	2	2
0	0	0	2	3	3
0	0	0	1	2	2



Observations

- We can make the following observations:
 1. There are **no deadlocks** in any state (at least one privilege is present).
 2. The **closure property** is satisfied (the system moves from a legal state to a legal state).
 3. **No starvation** (each machine has a chance of making more than 1 move).
 4. **Reachability** (there are always a series of moves to reach from one legal state to other).
- **All four constraints for a legitimate state are satisfied. So the system is stabilized.**

Few Other Works in the Field of Self Stabilization

Author	Year	Algorithm
E.W. Dijkstra	1974	Proposed the idea of self-stabilization
E. J. H. Chang et al.	1987	Costs of self-stabilization.
S. Aggarwal	1993, 1994	Time optimal self-stabilizing spanning tree algorithms
A. Arora and M. G. Gouda	1994	Distributed reset
G. Antonioiv and P.K. Srimani	1995, 1997, 1998	Self-stabilizing algorithms for minimum spanning trees construction
H. Kakugawa	2000	Universal Self-Stabilizing Mutual Exclusion Algorithm
F.C. Gartner	2003	Survey of algorithms for construction of self-stabilizing spanning trees.
Z. Shi et al.	2004	An anonymous self-stabilizing algorithm for 1-maximal independent set in trees.

Conclusion

- **Self-stabilization** has been used in many areas and the areas of study continue to grow. Algorithms have been developed **using central or distributed demons** and **uniform and non-uniform networks**.
- The algorithms that assume a central demon can usually be easily extended to support distributed demon, so these algorithms are still useful when applied to distributed systems.
- In this lecture, we have discussed the '**concept of self-stabilization**', '**system model**', '**related issues in the design of self-stabilizing algorithms and systems**' and also discuss '**Dijkstra's self-stabilizing token ring system**'